

# # Redis面试题

 面试鸭 mianshiya.com

## 程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

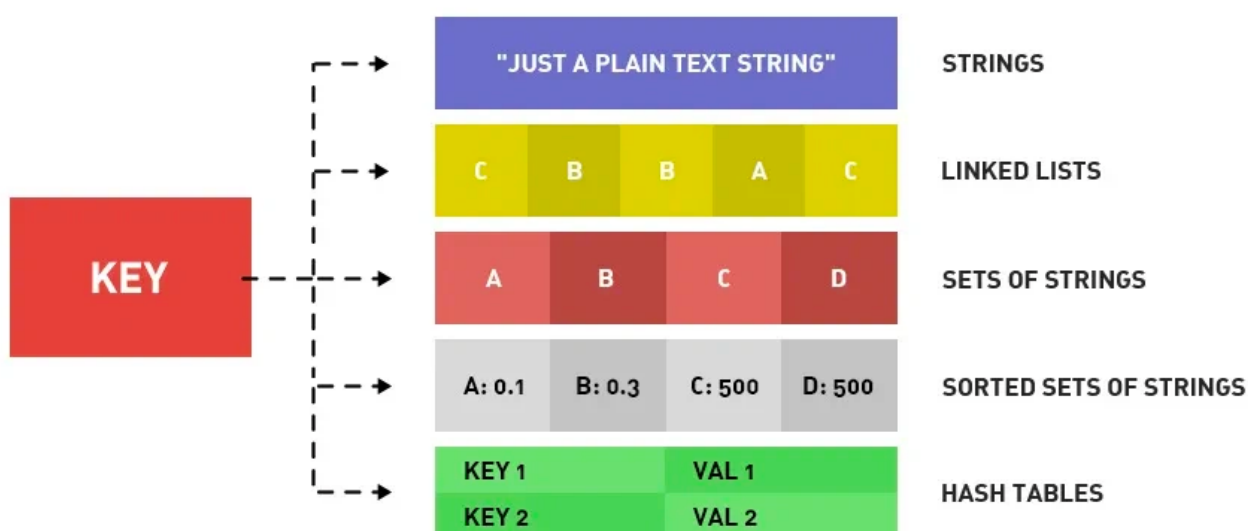
刷高频题拿高薪offer



## # 数据结构

### # 讲一下Redis底层的数据结构

Redis 提供了丰富的数据类型，常见的有五种数据类型：String（字符串），Hash（哈希），List（列表），Set（集合）、Zset（有序集合）。



| 结构类型       | 结构存储的值                | 结构的读写能力                                                              |
|------------|-----------------------|----------------------------------------------------------------------|
| String 字符串 | 可以是字符串、整数或浮点数         | 对整个字符串或字符串的一部分进行操作；对整数或浮点数进行自增或自减操作；                                 |
| List列表     | 一个链表，链表上的每个节点都包含一个字符串 | 对链表的两端进行push和pop操作，读取单个或多个元素；根据值查找或删除元素；                             |
| Set集合      | 包含字符串的无序集合            | 字符串的集合，包含基础的方法有看是否存在添加、获取、删除；还包含计算交集、并集、差集等                          |
| Hash散列     | 包含键值对的无序散列表           | 包含方法有添加、获取、删除单个元素                                                    |
| Zset有序集合   | 和散列一样，用于存储键值对         | 字符串成员与浮点数分数之间的有序映射；元素的排列顺序由分数的大小决定；包含方法有添加、获取、删除单个元素以及根据分值范围或成员来获取元素 |

随着 Redis 版本的更新，后面又支持了四种数据类型：**BitMap（2.2 版新增）、HyperLogLog（2.8 版新增）、GEO（3.2 版新增）、Stream（5.0 版新增）**。Redis 五种数据类型的应用场景：

- String 类型的应用场景：缓存对象、常规计数、分布式锁、共享 session 信息等。
- List 类型的应用场景：消息队列（但是有两个问题：1. 生产者需要自行实现全局唯一 ID；2. 不能以消费组形式消费数据）等。
- Hash 类型：缓存对象、购物车等。
- Set 类型：聚合计算（并集、交集、差集）场景，比如点赞、共同关注、抽奖活动等。
- Zset 类型：排序场景，比如排行榜、电话和姓名排序等。

Redis 后续版本又支持四种数据类型，它们的应用场景如下：

- BitMap（2.2 版新增）：二值状态统计的场景，比如签到、判断用户登陆状态、连续签到用户总数等；
- HyperLogLog（2.8 版新增）：海量数据基数统计的场景，比如百万级网页 UV 计数等；
- GEO（3.2 版新增）：存储地理位置信息的场景，比如滴滴叫车；
- Stream（5.0 版新增）：消息队列，相比于基于 List 类型实现的消息队列，有这两个特有的特性：自动生成全局唯一消息ID，支持以消费组形式消费数据。

## # ZSet用过吗

用过 zset 实现排行榜的功能。

以博文点赞排名为例，小林发表了五篇博文，分别获得赞为 200、40、100、50、150。

```
# article:1 文章获得了200个赞
> ZADD user:xiaolin:ranking 200 article:1
(integer) 1
# article:2 文章获得了40个赞
> ZADD user:xiaolin:ranking 40 article:2
(integer) 1
# article:3 文章获得了100个赞
> ZADD user:xiaolin:ranking 100 article:3
(integer) 1
# article:4 文章获得了50个赞
> ZADD user:xiaolin:ranking 50 article:4
(integer) 1
# article:5 文章获得了150个赞
> ZADD user:xiaolin:ranking 150 article:5
```

```
(integer) 1
```

文章 article:4 新增一个赞，可以使用 ZINCRBY 命令（为有序集合key中元素member的分值加上increment）：

```
> ZINCRBY user:xiaolin:ranking 1 article:4
"51"
```

查看某篇文章的赞数，可以使用 ZSCORE 命令（返回有序集合key中元素个数）：

```
> ZSCORE user:xiaolin:ranking article:4
"50"
```

获取小林文章赞数最多的 3 篇文章，可以使用 ZREVRANGE 命令（倒序获取有序集合 key 从start下标到stop下标的元素）：

```
# WITHSCORES 表示把 score 也显示出来
> ZREVRANGE user:xiaolin:ranking 0 2 WITHSCORES
1) "article:1"
2) "200"
3) "article:5"
4) "150"
5) "article:3"
6) "100"
```

获取小林 100 赞到 200 赞的文章，可以使用 ZRANGEBYSCORE 命令（返回有序集合中指定分数区间内的成员，分数由低到高排序）：

```
> ZRANGEBYSCORE user:xiaolin:ranking 100 200 WITHSCORES
1) "article:3"
2) "100"
3) "article:5"
4) "150"
5) "article:1"
6) "200"
```

## # Zset 底层是怎么实现的？

Zset 类型的底层数据结构是由**压缩列表或跳表**实现的：

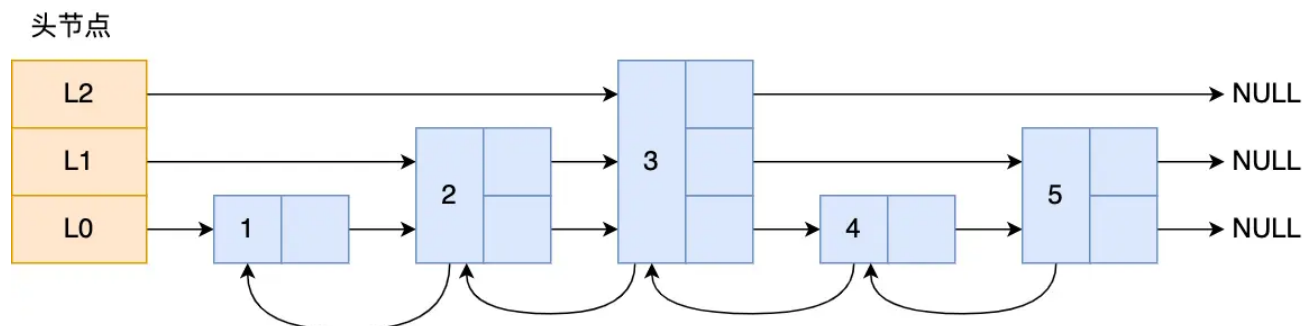
- 如果有序集合的元素个数小于 128 个，并且每个元素的值小于 64 字节时，Redis 会使用**压缩列表**作为 Zset 类型的底层数据结构；
- 如果有序集合的元素不满足上面的条件，Redis 会使用**跳表**作为 Zset 类型的底层数据结构；

在 Redis 7.0 中，压缩列表数据结构已经废弃了，交由 listpack 数据结构来实现了。

## # 跳表是怎么实现的？

链表在查找元素的时候，因为需要逐一查找，所以查询效率非常低，时间复杂度是 $O(N)$ ，于是就出现了跳表。**跳表是在链表基础上改进过来的，实现了一种「多层」的有序链表**，这样的好处是能快速定位数据。

那跳表长什么样呢？我这里举个例子，下图展示了一个层级为 3 的跳表。



图中头节点有 L0~L2 三个头指针，分别指向了不同层级的节点，然后每个层级的节点都通过指针连接起来：

- L0 层级共有 5 个节点，分别是节点 1、2、3、4、5；
- L1 层级共有 3 个节点，分别是节点 2、3、5；
- L2 层级只有 1 个节点，也就是节点 3。

如果我们要在链表中查找节点 4 这个元素，只能从头开始遍历链表，需要查找 4 次，而使用了跳表后，只需要查找 2 次就能定位到节点 4，因为可以在头节点直接从 L2 层级跳到节点 3，然后再往前遍历找到节点 4。

可以看到，这个查找过程就是在多个层级上跳来跳去，最后定位到元素。当数据量很大时，跳表的查找复杂度就是  $O(\log N)$ 。

那跳表节点是怎么实现多层级的呢？这就需要看「跳表节点」的数据结构了，如下：

```
typedef struct zskiplistNode {
    //Zset 对象的元素值
    sds ele;
    //元素权重值
    double score;
    //后向指针
    struct zskiplistNode *backward;

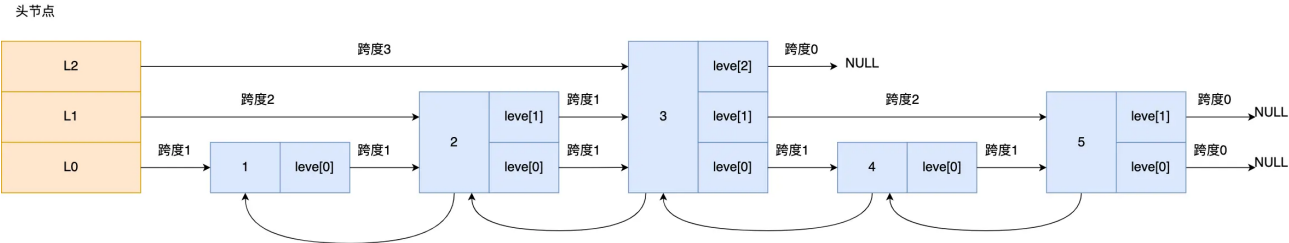
    //节点的level数组，保存每层上的前向指针和跨度
    struct zskiplistLevel {
        struct zskiplistNode *forward;
        unsigned long span;
    } level[];
} zskiplistNode;
```

Zset 对象要同时保存「元素」和「元素的权重」，对应到跳表节点结构里就是 sds 类型的 ele 变量和 double 类型的 score 变量。每个跳表节点都有一个后向指针（struct zskiplistNode \*backward），指向前一个节点，目的是为了从跳表的尾节点开始访问节点，这样倒序查找时很方便。

跳表是一个带有层级关系的链表，而且每一层级可以包含多个节点，每一个节点通过指针连接起来，实现这一特性就是靠跳表节点结构体中的 **zskiplistLevel 结构体类型的 level 数组**。

level 数组中的每一个元素代表跳表的一层，也就是由 zskiplistLevel 结构体表示，比如 level[0] 就表示第一层，level[1] 就表示第二层。zskiplistLevel 结构体里定义了「指向下一个跳表节点的指针」和「跨度」，跨度时用来记录两个节点之间的距离。

比如，下面这张图，展示了各个节点的跨度。



第一眼看到跨度的时候，以为是遍历操作有关，实际上并没有任何关系，遍历操作只需要用前向指针（struct zskiplistNode \*forward）就可以完成了。

Redis 跳表在创建节点的时候，随机生成每个节点的层数，并没有严格维持相邻两层的节点数量比例为 2 : 1 的情况。

具体的做法是，跳表在创建节点时候，会生成范围为[0-1]的一个随机数，如果这个随机数小于 0.25（相当于概率 25%），那么层数就增加 1 层，然后继续生成下一个随机数，直到随机数的结果大于 0.25 结束，最终确定该节点的层数。

这样的做法，相当于每增加一层的概率不超过 25%，层数越高，概率越低，层高最大限制是 64。

虽然我前面讲解跳表的时候，图中的跳表的「头节点」都是 3 层高，但是其实如果层高最大限制是 64，那么在创建跳表「头节点」的时候，就会直接创建 64 层高的头节点。

## # 跳表是怎么设置层高的？

跳表在创建节点时候，会生成范围为[0-1]的一个随机数，如果这个随机数小于 0.25（相当于概率 25%），那么层数就增加 1 层，然后继续生成下一个随机数，直到随机数的结果大于 0.25 结束，最终确定该节点的层数。

## # Redis为什么使用跳表而不是用B+树？

Redis 是内存数据库，跳表在实现简单性、写入性能、内存访问模式等方面的综合优势，使其成为更合适的选择。

| 维度    | 跳表优势               | B+ 树劣势           |
|-------|--------------------|------------------|
| 内存访问  | 符合CPU缓存局部性，指针跳转更高效 | 节点结构复杂，缓存不友好     |
| 实现复杂度 | 代码简洁，无复杂平衡操作       | 节点分裂/合并逻辑复杂，代码量大 |
| 写入性能  | 插入/删除仅需调整局部指针      | 插入可能触发递归节点分裂，成本高 |
| 内存占用  | 结构紧凑，无内部碎片         | 节点预分配可能浪费内存      |

Redis 选择使用跳表（Skip List）而不是 B+ 树来实现有序集合（Sorted Set）等数据结构，是经过多方面权衡后的结果。以下是详细的原因分析：

### 1、内存结构与访问模式的差异

#### B+ 树的特性

- **磁盘友好**：B+ 树的设计目标是优化磁盘 I/O，通过减少树的高度来降低磁盘寻道次数（例如，一个 3 层的 B+ 树可以管理数百万数据）。
- **节点填充率高**：每个节点存储多个键值（Page/Block），适合批量读写。

- **范围查询高效**：叶子节点形成有序链表，范围查询（如 `ZRANGE`）性能极佳。

跳表的特性

- **内存友好**：跳表基于链表，通过多级索引加速查询，**内存访问模式更符合CPU缓存局部性**（指针跳跃更少）。
- **简单灵活**：插入/删除时仅需调整局部指针，无需复杂的节点分裂与合并。
- **概率平衡**：通过随机层高实现近似平衡，避免了严格的平衡约束（如红黑树的旋转）。

Redis 是内存数据库，数据完全存储在内存中，不需要优化磁盘 I/O，因此 B+ 树的磁盘友好特性对 Redis 意义不大。而跳表的内存访问模式更优，更适合高频的内存操作。

2、实现复杂度的对比

B+ 树的实现复杂度：

- **节点分裂与合并**：插入/删除时可能触发节点分裂或合并，需要复杂的再平衡逻辑。
- **锁竞争**：在并发环境下，B+ 树的锁粒度较粗（如页锁），容易成为性能瓶颈。
- **代码复杂度**：B+ 树的实现需要处理大量边界条件（如最小填充因子、兄弟节点借用等）。

跳表的实现复杂度：

- **无再平衡操作**：插入时只需随机生成层高，删除时直接移除节点并调整指针。
- **细粒度锁或无锁**：跳表可以通过分段锁或无锁结构（如 CAS）实现高效并发。
- **代码简洁**：Redis 的跳表核心代码仅需约 200 行（B+ 树实现通常需要数千行）。

对于 Redis 这种追求高性能和代码简洁性的项目，跳表的低实现复杂度更具吸引力，Redis 作者 Antirez 曾表示，跳表的实现复杂度远低于平衡树，且性能相近，是更优选择。

3、性能对比

查询性能

- **单点查询**：跳表和 B+ 树的时间复杂度均为  $O(\log N)$ ，但跳表的实际常数更小（内存中指针跳转比磁盘块访问快得多）。
- **范围查询**：B+ 树的叶子链表在范围查询时占优，但跳表通过双向链表也能高效支持 `ZRANGE` 操作。

写入性能

- **B+ 树**：插入可能触发节点分裂，涉及父节点递归更新，成本较高。
- **跳表**：插入仅需修改相邻节点的指针，写入性能更优（Redis 的 `ZADD` 操作时间复杂度为  $O(\log N)$ ）。

实测数据：在内存中，跳表的插入速度比 B+ 树快 2-3 倍，查询速度相当。

4、内存占用

- **B+ 树**：每个节点需要存储多个键值和子节点指针，存在内部碎片（节点未填满时）。
- **跳表**：每个节点只需存储键值、层高和多个前向指针，内存占用更紧凑。

# 压缩列表是怎么实现的？

压缩列表是 Redis 为了节约内存而开发的，它是由连续内存块组成的顺序型数据结构，有点类似于数组。

|         |        |       |        |        |     |        |       |
|---------|--------|-------|--------|--------|-----|--------|-------|
| zlbytes | zltail | zllen | entry1 | entry2 | ... | entryN | zlend |
|---------|--------|-------|--------|--------|-----|--------|-------|

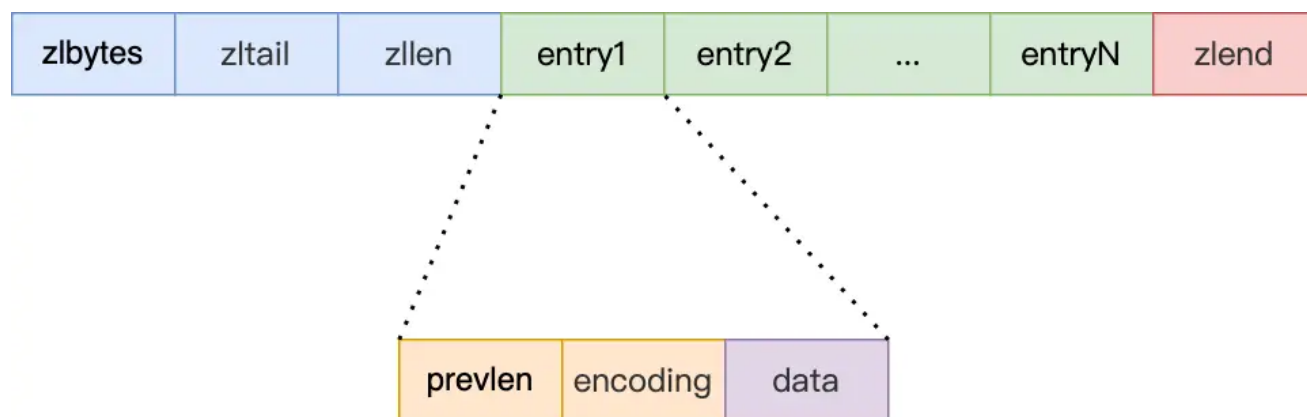
压缩列表在表头有三个字段：



- **\*zlbytes\***，记录整个压缩列表占用对内存字节数；
- **\*zltail\***，记录压缩列表「尾部」节点距离起始地址由多少字节，也就是列表尾的偏移量；
- **\*zllen\***，记录压缩列表包含的节点数量；
- **\*zlend\***，标记压缩列表的结束点，固定值 0xFF（十进制255）。

在压缩列表中，如果我们要查找定位第一个元素和最后一个元素，可以通过表头三个字段（zllen）的长度直接定位，复杂度是  $O(1)$ 。而**查找其他元素时，就没有这么高效了，只能逐个查找，此时的复杂度就是  $O(N)$  了，因此压缩列表不适合保存过多的元素。**

另外，压缩列表节点（entry）的构成如下：



压缩列表节点包含三部分内容：

- **prevlen**，记录了「前一个节点」的长度，目的是为了实现从后向前遍历；
- **encoding**，记录了当前节点实际数据的「类型和长度」，类型主要有两种：字符串和整数。
- **data**，记录了当前节点的实际数据，类型和长度都由 encoding 决定；

当我们往压缩列表中插入数据时，压缩列表就会根据数据类型是字符串还是整数，以及数据的大小，会使用不同空间大小的 prevlen 和 encoding 这两个元素里保存的信息，**这种根据数据大小和类型进行不同的空间大小分配的设计思想，正是 Redis 为了节省内存而采用的。**

压缩列表的缺点是会发生连锁更新的问题，因此**连锁更新一旦发生，就会导致压缩列表占用的内存空间要多次重新分配，这就会直接影响到压缩列表的访问性能。**

所以说，**虽然压缩列表紧凑型的内存布局能节省内存开销，但是如果保存的元素数量增加了，或是元素变大了，会导致内存重新分配，最糟糕的是会有「连锁更新」的问题。**

因此，**压缩列表只会用于保存的节点数量不多的场景**，只要节点数量足够小，即使发生连锁更新，也是能接受的。

虽说如此，Redis 针对压缩列表在设计上的不足，在后来的版本中，新增设计了两种数据结构：quicklist（Redis 3.2 引入）和 listpack（Redis 5.0 引入）。这两种数据结构的设计目标，就是尽可能地保持压缩列表节省内存的优势，同时解决压缩列表的「连锁更新」的问题。

## # 介绍一下 Redis 中的 listpack

quicklist 虽然通过控制 quicklistNode 结构里的压缩列表的大小或者元素个数，来减少连锁更新带来的性能影响，但是并没有完全解决连锁更新的问题。

因为 quicklistNode 还是用了压缩列表来保存元素，压缩列表连锁更新的问题，来源于它的结构设计，所以要想彻底解决这个问题，需要设计一个新的数据结构。

于是，Redis 在 5.0 新设计一个数据结构叫 listpack，目的是替代压缩列表，它最大特点是 listpack 中每个节点不再包含前一个节点的长度了，压缩列表每个节点正因为需要保存前一个节点的长度字段，就会有连锁更新的隐患。

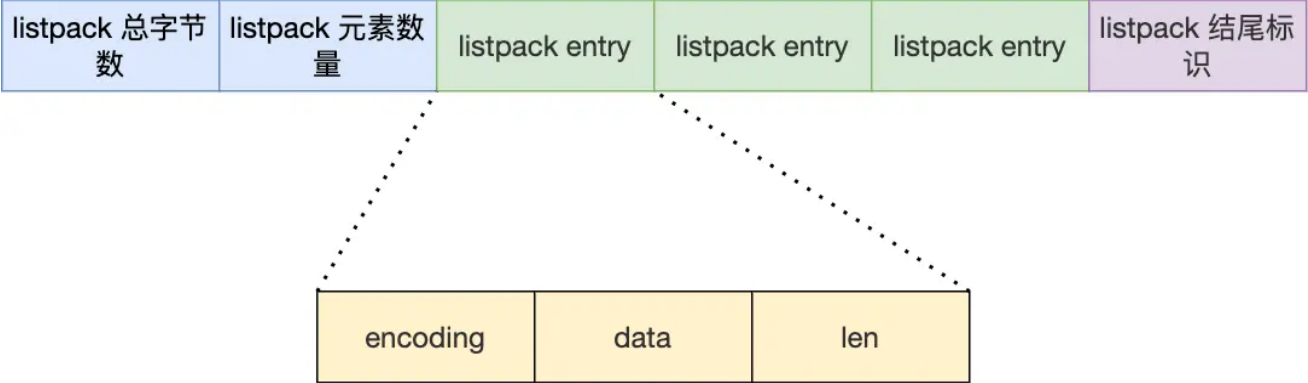
listpack 采用了压缩列表的很多优秀的设计，比如还是用一块连续的内存空间来紧凑地保存数据，并且为了节省内存的开销，listpack 节点会采用不同的编码方式保存不同大小的数据。

我们先看看 listpack 结构：



listpack 头包含两个属性，分别记录了 listpack 总字节数和元素数量，然后 listpack 末尾也有个结尾标识。图中的 listpack entry 就是 listpack 的节点了。

每个 listpack 节点结构如下：



主要包含三方面内容：

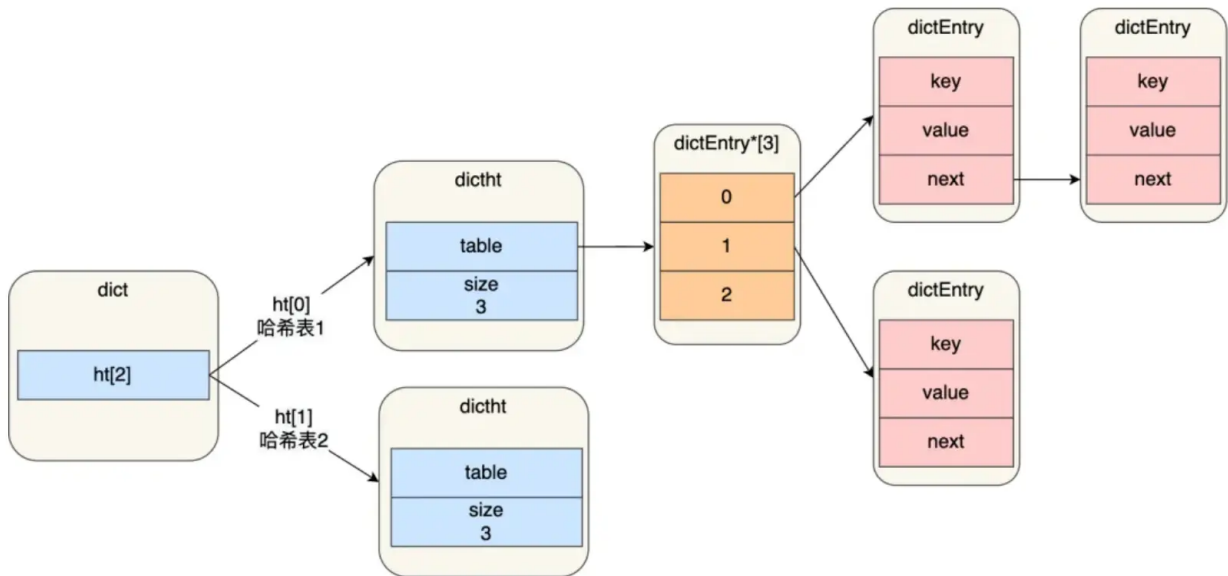
- encoding，定义该元素的编码类型，会对不同长度的整数和字符串进行编码；
- data，实际存放的数据；
- len，encoding+data的总长度；

可以看到，listpack 没有压缩列表中记录前一个节点长度的字段了，listpack 只记录当前节点的长度，当我们向 listpack 加入一个新元素的时候，不会影响其他节点的长度字段的变化，从而避免了压缩列表的连锁更新问题。

## # 哈希表是怎么扩容的？

进行 rehash 的时候，需要用上 2 个哈希表了。

扩容的时候，需要用到 2 个哈希表。



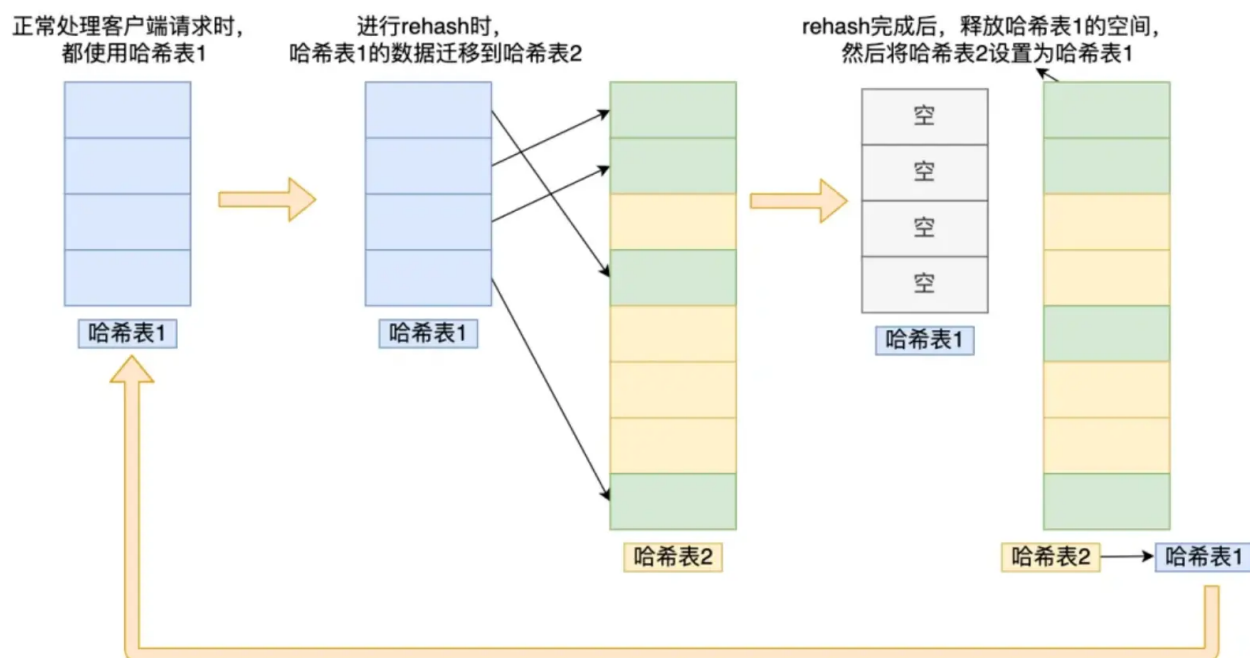


在正常服务请求阶段，插入的数据，都会写入到「哈希表 1」，此时的「哈希表 2」并没有被分配空间。

随着数据逐步增多，触发了 rehash 操作，这个过程分为三步：

- 给「哈希表 2」分配空间，一般会比「哈希表 1」大 2 倍；
- 将「哈希表 1」的数据迁移到「哈希表 2」中；
- 迁移完成后，「哈希表 1」的空间会被释放，并把「哈希表 2」设置为「哈希表 1」，然后在「哈希表 2」新创建一个空白的哈希表，为下次 rehash 做准备。

为了方便你理解，我把 rehash 这三个过程画在了下面这张图：



这个过程看起来简单，但是其实第二步很有问题，如果「哈希表 1」的数据量非常大，那么在迁移至「哈希表 2」的时候，因为会涉及大量的数据拷贝，此时可能会对 Redis 造成阻塞，无法服务其他请求。

为了避免 rehash 在数据迁移过程中，因拷贝数据的耗时，影响 Redis 性能的情况，所以 Redis 采用了**渐进式 rehash**，也就是将数据的迁移的工作不再是一次性迁移完成，而是分多次迁移。

渐进式 rehash 步骤如下：

- 给「哈希表 2」分配空间；
- 在 rehash 进行期间，每次哈希表元素进行新增、删除、查找或者更新操作时，Redis 除了会执行对应的操作之外，还会顺序将「哈希表 1」中索引位置上的所有 key-value 迁移到「哈希表 2」上；
- 随着处理客户端发起的哈希表操作请求数量越多，最终在某个时间点会把「哈希表 1」的所有 key-value 迁移到「哈希表 2」，从而完成 rehash 操作。

这样就巧妙地把一次性大量数据迁移工作的开销，分摊到了多次处理请求的过程中，避免了一次性 rehash 的耗时操作。

在进行渐进式 rehash 的过程中，会有两个哈希表，所以在渐进式 rehash 进行期间，哈希表元素的删除、查找、更新等操作都会在这两个哈希表进行。比如，查找一个 key 的值的话，先会在「哈希表 1」里面进行查找，如果没找到，就会继续到哈希表 2 里面进行找到。

另外，在渐进式 rehash 进行期间，新增一个 key-value 时，会被保存到「哈希表 2」里面，而「哈希表 1」则不再进行任何添加操作，这样保证了「哈希表 1」的 key-value 数量只会减少，随着 rehash 操作的完成，最终「哈希表 1」就会变成空表。

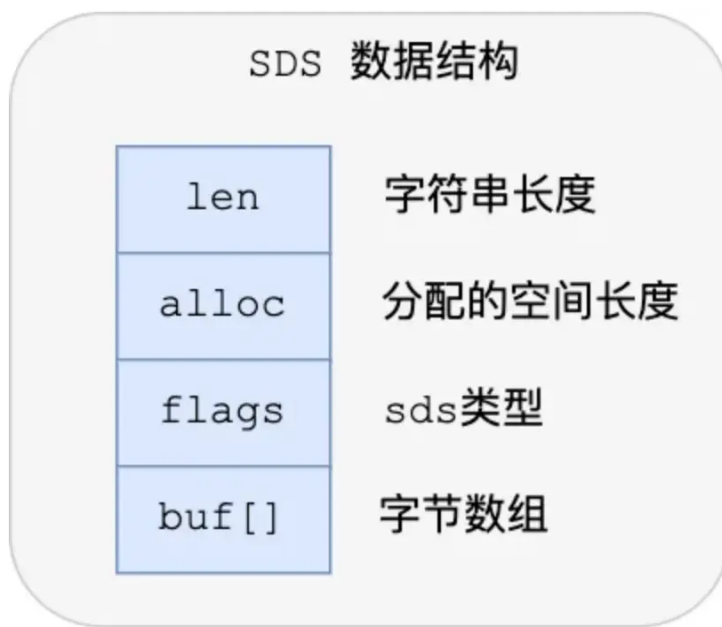
## # 哈希表扩容的时候，有读请求怎么查？

查找一个 key 的值的话，先会在「哈希表 1」 里面进行查找，如果没找到，就会继续到哈希表 2 里面进行找到。

## # String 是使用什么存储的？为什么不用 c 语言中的字符串？

Redis 的 String 字符串是用 SDS 数据结构存储的。

下图就是 Redis 5.0 的 SDS 的数据结构：



结构中的每个成员变量分别介绍下：

- **len，记录了字符串长度。**这样获取字符串长度的时候，只需要返回这个成员变量值就行，时间复杂度只需要  $O(1)$ 。
- **alloc，分配给字符数组的空间长度。**这样在修改字符串的时候，可以通过 `alloc - len` 计算出剩余的空间大小，可以用来判断空间是否满足修改需求，如果不满足的话，就会自动将 SDS 的空间扩展至执行修改所需的大小，然后才执行实际的修改操作，所以使用 SDS 既不需要手动修改 SDS 的空间大小，也不会出现前面所说的缓冲区溢出的问题。
- **flags，用来表示不同类型的 SDS。**一共设计了 5 种类型，分别是 `sdshdr5`、`sdshdr8`、`sdshdr16`、`sdshdr32` 和 `sdshdr64`，后面在说明区别之处。
- **buf[]，字符数组，用来保存实际数据。**不仅可以保存字符串，也可以保存二进制数据。

总的来说，Redis 的 SDS 结构在原本字符数组之上，增加了三个元数据：`len`、`alloc`、`flags`，用来解决 C 语言字符串的缺陷。

### 复杂度获取字符串长度

C 语言的字符串长度获取 `strlen` 函数，需要通过遍历的方式来统计字符串长度，时间复杂度是  $O(N)$ 。

而 Redis 的 SDS 结构因为加入了 `len` 成员变量，那么**获取字符串长度的时候，直接返回这个成员变量的值就行，所以复杂度只有  $O(1)$ 。**

### 二进制安全

因为 SDS 不需要用“\0”字符来标识字符串结尾了，而是**有个专门的 len 成员变量来记录长度，所以可存储包含“\0”的数据。**但是 SDS 为了兼容部分 C 语言标准库的函数，SDS 字符串结尾还是会加上“\0”字符。

因此，SDS 的 API 都是以处理二进制的方式来处理 SDS 存放在 buf[] 里的数据，程序不会对其中的数据做任何限制，数据写入的时候时什么样的，它被读取时就是什么样的。

通过使用二进制安全的 SDS，而不是 C 字符串，使得 Redis 不仅可以保存文本数据，也可以保存任意格式的二进制数据。

不会发生缓冲区溢出

C 语言的字符串标准库提供的字符串操作函数，大多数（比如 strcat 追加字符串函数）都是不安全的，因为这些函数把缓冲区大小是否满足操作需求的工作交由开发者来保证，程序内部并不会判断缓冲区大小是否足够用，当发生了缓冲区溢出就有可能造成程序异常结束。

所以，Redis 的 SDS 结构里引入了 alloc 和 len 成员变量，这样 SDS API 通过 `alloc - len` 计算，可以算出剩余可用的空间大小，这样在对字符串做修改操作的时候，就可以由程序内部判断缓冲区大小是否足够用。

而且，当判断出缓冲区大小不够用时，Redis 会自动将扩大 SDS 的空间大小，以满足修改所需的大小。

## # 线程模型

### # Redis为什么快？

官方使用基准测试的结果是，**单线程的 Redis 吞吐量可以达到 10W/每秒**，如下图所示：



之所以 Redis 采用单线程（网络 I/O 和执行命令）那么快，有如下几个原因：

- Redis 的大部分操作**都在内存中完成**，并且采用了高效的数据结构，因此 Redis 瓶颈可能是机器的内存或者网络带宽，而并非 CPU，既然 CPU 不是瓶颈，那么自然就采用单线程的解决方案了；
- Redis 采用单线程模型可以**避免了多线程之间的竞争**，省去了多线程切换带来的时间和性能上的开销，而且也不会导致死锁问题。
- Redis 采用了 **I/O 多路复用机制**处理大量的客户端 Socket 请求，IO 多路复用机制是指一个线程处理多个 IO 流，就是我们经常听到的 select/epoll 机制。简单来说，在 Redis 只运行单线程的情况下，该机制允许内核

中，同时存在多个监听 Socket 和已连接 Socket。内核会一直监听这些 Socket 上的连接请求或数据请求。一旦有请求到达，就会交给 Redis 线程处理，这就实现了一个 Redis 线程处理多个 IO 流的效果。

## # Redis哪些地方使用了多线程？

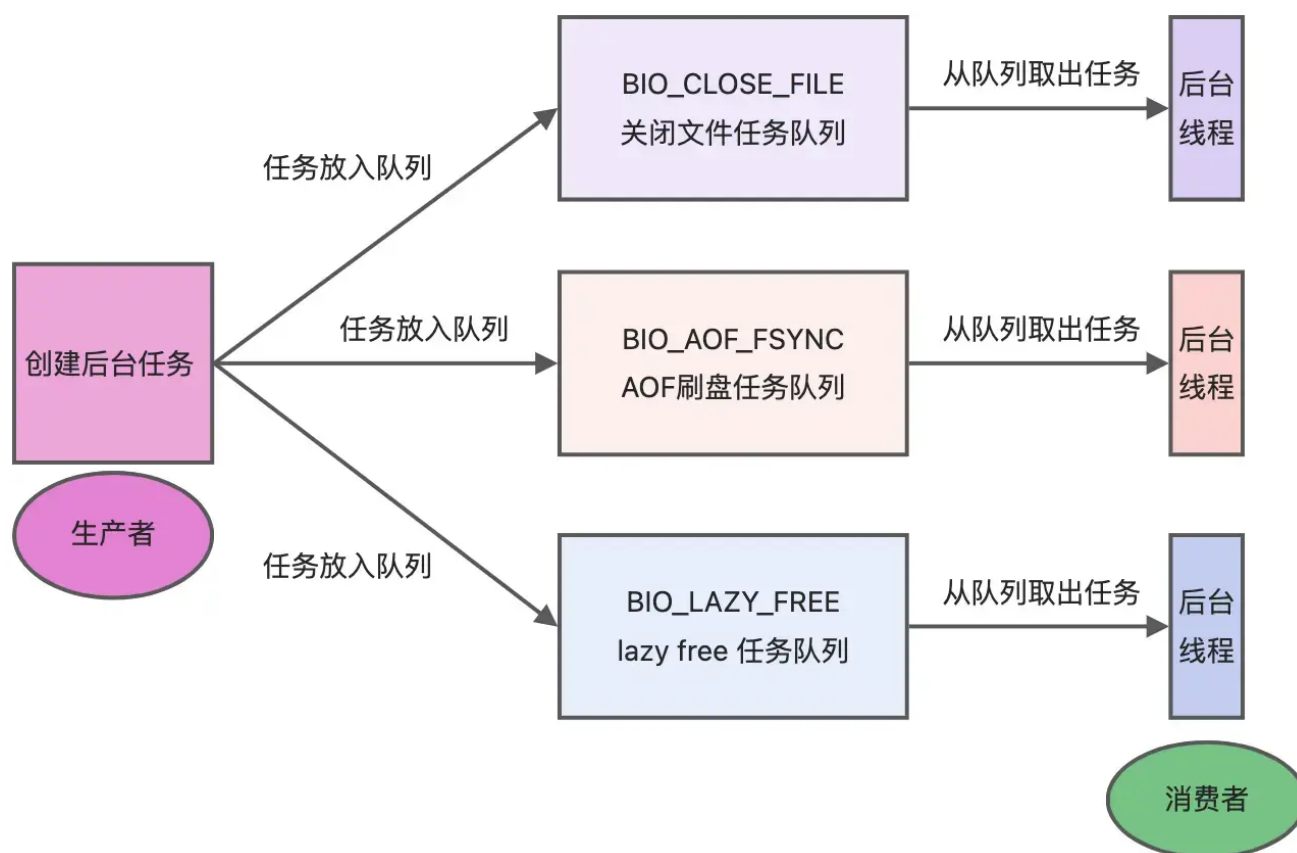
Redis 单线程指的是「接收客户端请求->解析请求->进行数据读写等操作->发送数据给客户端」这个过程是由一个线程（主线程）来完成的，这也是我们常说 Redis 是单线程的原因。

但是，Redis 程序并不是单线程的，Redis 在启动的时候，是会启动后台线程（BIO）的：

- Redis 在 2.6 版本，会启动 2 个后台线程，分别处理关闭文件、AOF 刷盘这两个任务；
- Redis 在 4.0 版本之后，新增了一个新的后台线程，用来异步释放 Redis 内存，也就是 lazyfree 线程。例如执行 unlink key / flushdb async / flushall async 等命令，会把这些删除操作交给后台线程来执行，好处是会导致 Redis 主线程卡顿。因此，当我们要删除一个大 key 的时候，不要使用 del 命令删除，因为 del 是在主线程处理的，这样会导致 Redis 主线程卡顿，因此我们应该使用 unlink 命令来异步删除大key。

之所以 Redis 为「关闭文件、AOF 刷盘、释放内存」这些任务创建单独的线程来处理，是因为这些任务的操作都是很耗时的，如果把这些任务都放在主线程来处理，那么 Redis 主线程就很容易发生阻塞，这样就无法处理后续的请求了。

后台线程相当于一个消费者，生产者把耗时任务丢到任务队列中，消费者（BIO）不停轮询这个队列，拿出任务就去执行对应的方法即可。



虽然 Redis 的主要工作（网络 I/O 和执行命令）一直是单线程模型，但是在 Redis 6.0 版本之后，也采用了多个 I/O 线程来处理网络请求，这是因为随着网络硬件的性能提升，Redis 的性能瓶颈有时会出现网络 I/O 的处理上。

所以为了提高网络 I/O 的并行度，Redis 6.0 对于网络 I/O 采用多线程来处理。但是对于命令的执行，Redis 仍然使用单线程来处理，所以大家不要误解 Redis 有多线程同时执行命令。

Redis 官方表示，**Redis 6.0 版本引入的多线程 I/O 特性对性能提升至少是一倍以上。**

Redis 6.0 版本支持的 I/O 多线程特性，默认情况下 I/O 多线程只针对发送响应数据（write client socket），并不会以多线程的方式处理读请求（read client socket）。要想开启多线程处理客户端读请求，就需要把 Redis.conf 配置文件中的 io-threads-do-reads 配置项设为 yes。

```
//读请求也使用io多线程
io-threads-do-reads yes
```

同时，Redis.conf 配置文件中提供了 IO 多线程个数的配置项。

```
// io-threads N, 表示启用 N-1 个 I/O 多线程（主线程也算一个 I/O 线程）
io-threads 4
```

关于线程数的设置，官方的建议是如果为 4 核的 CPU，建议线程数设置为 2 或 3，如果为 8 核 CPU 建议线程数设置为 6，线程数一定要小于机器核数，线程数并不是越大越好。

因此，Redis 6.0 版本之后，Redis 在启动的时候，默认情况下会**额外创建 6 个线程**（这里的线程数不包括主线程）：

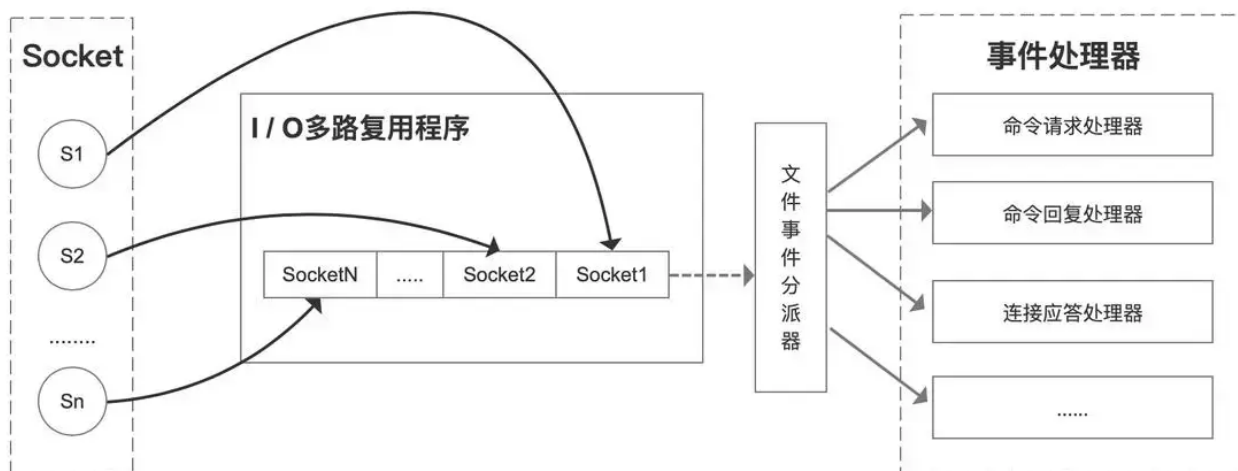
- Redis-server：Redis 的主线程，主要负责执行命令；
- bio\_close\_file、bio\_aof\_fsync、bio\_lazy\_free：三个后台线程，分别异步处理关闭文件任务、AOF 刷盘任务、释放内存任务；
- io\_thd\_1、io\_thd\_2、io\_thd\_3：三个 I/O 线程，io-threads 默认是 4，所以会启动 3（4-1）个 I/O 多线程，用来分担 Redis 网络 I/O 的压力。

## # Redis怎么实现的io多路复用？

为什么 Redis 中要使用 I/O 多路复用这种技术呢？

因为 Redis 是跑在「单线程」中的，所有的操作都是按照顺序线性执行的，但是由于读写操作等待用户输入或输出都是阻塞的，所以 I/O 操作在一般情况下往往不能直接返回，这会导致某一文件的 I/O 阻塞，导致整个进程无法对其它客户提供服务。而 I/O 多路复用就是为了解决这个问题而出现的。为了让单线程(进程)的服务端应用同时处理多个客户端的事件，Redis 采用了 IO 多路复用机制。

这里“多路”指的是多个网络连接客户端，“复用”指的是复用同一个线程(单进程)。I/O 多路复用其实是使用一个线程来检查多个 Socket 的就绪状态，在单个线程中通过记录跟踪每一个 socket（I/O 流）的状态来管理处理多个 I/O 流。如下图所示是 Redis 的 I/O 多路复用模型：



如上图对 Redis 的 I/O 多路复用模型进行一下描述说明：

- 一个 socket 客户端与服务端连接时，会生成对应一个套接字描述符(套接字描述符是文件描述符的一种)，每一个 socket 网络连接其实都对应一个文件描述符。
- 多个客户端与服务端连接时，Redis 使用 I/O 多路复用程序 将客户端 socket 对应的 FD 注册到监听列表(一个队列)中。当服务端执行 read、write 等操作命令时，I/O 多路复用程序会将命令封装成一个事件，并绑定到对应的 FD 上。
- 文件事件处理器使用 I/O 多路复用模块同时监控多个文件描述符 (fd) 的读写情况，当 accept、read、write 和 close 文件事件产生时，文件事件处理器就会回调 FD 绑定的事件处理器进行处理相关命令操作。

例如：以 Redis 的 I/O 多路复用程序 epoll 函数为例。多个客户端连接服务端时，Redis 会将客户端 socket 对应的 fd 注册进 epoll，然后 epoll 同时监听多个文件描述符(FD)是否有数据到来，如果有数据来了就通知事件处理器赶紧处理，这样就不会存在服务端一直等待某个客户端给数据的情形。

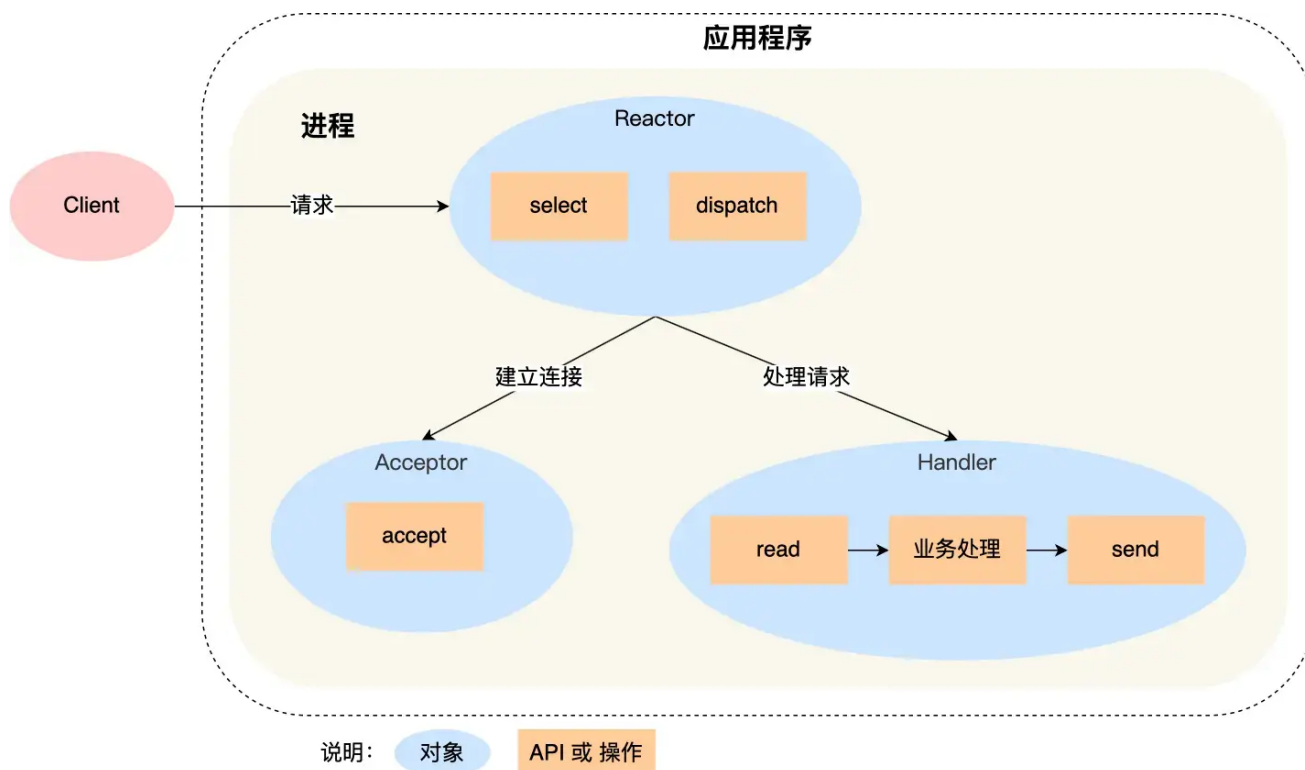
整个文件事件处理器是在单线程上运行的，但是通过 I/O 多路复用模块的引入，实现了同时对多个 FD 读写的监控，当其中一个 client 端达到写或读的状态，文件事件处理器就马上执行，从而就不会出现 I/O 堵塞的问题，提高了网络通信的性能。

Redis 的 I/O 多路复用模式使用的是 Reactor 设置模式的方式来实现。

## # Redis的网络模型是怎样的？

Redis 6.0 版本之前，用的是单Reactor单线程的模式





单 Reactor 单进程的方案因为全部工作都在同一个进程内完成，所以实现起来比较简单，不需要考虑进程间通信，也不用担心多进程竞争。

但是，这种方案存在 2 个缺点：

- 第一个缺点，因为只有一个进程，**无法充分利用多核 CPU 的性能**；
- 第二个缺点，Handler 对象在业务处理时，整个进程是无法处理其他连接的事件的，**如果业务处理耗时比较长，那么就造成响应的延迟**；

所以，单 Reactor 单进程的方案**不适用计算机密集型的场景，只适用于业务处理非常快速的场景**。

Redis 是由 C 语言实现的，在 Redis 6.0 版本之前采用的正是「单 Reactor 单进程」的方案，因为 Redis 业务处理主要是在内存中完成，操作的速度是很快的，性能瓶颈不在 CPU 上，所以 Redis 对于命令的处理是单进程的方案。

**到 Redis 6.0 之后，就将网络 I/O 的处理改成多线程的方式了，目的是为了这是因为随着网络硬件的性能提升，Redis 的性能瓶颈有时会出现网络 I/O 的处理上。**

所以为了提高网络 I/O 的并行度，Redis 6.0 对于网络 I/O 采用多线程来处理。**但是对于命令的执行，Redis 仍然使用单线程来处理，所以大家不要误解 Redis 有多线程同时执行命令。**

## # 事务

### # 如何实现redis 原子性？

redis 执行一条命令的时候是具备原子性的，因为 redis 执行命令的时候是单线程来处理的，不存在多线程安全的问题。

如果要保证 2 条命令的原子性的话，可以考虑用 lua 脚本，将多个操作写到一个 Lua 脚本中，Redis 会把整个 Lua 脚本作为一个整体执行，在执行的过程中不会被其他命令打断，从而保证了 Lua 脚本中操作的原子性。

比如说，在用 redis 实现分布式锁的场景下，解锁期间涉及 2 个操作，分别是先判断锁是不是自己的，是自己的才能删除锁，为了保证这 2 个操作的原子性，会通过 lua 脚本来保证原子性。

```
// 释放锁时，先比较 unique_value 是否相等，避免锁的误释放
if redis.call("get",KEYS[1]) == ARGV[1] then
    return redis.call("del",KEYS[1])
else
    return 0
end
```

## # 除了lua有没有什么也能保证redis的原子性？

redis 事务也可以保证多个操作的原子性。

如果 redis 事务正常执行，没有发生任何错误，那么使用 MULTI 和 EXEC 配合使用，就可以保证多个操作都完成。

但是，如果事务执行发生错误了，就没办法保证原子性了。比如说 2 个操作，第一个操作执行成果了，但是第二个操作执行的时候，命令出错了，那事务并不会回滚，因为Redis 中并没有提供回滚机制。

举个小例子。事务中的 LPOP 命令对 String 类型数据进行操作，入队时没有报错，但是，在 EXEC 执行时报错了。LPOP 命令本身没有执行成功，但是事务中的 DECR 命令却成功执行了。

```
#开启事务
127.0.0.1:6379> MULTI
OK
#发送事务中的第一个操作，LPOP命令操作的数据类型不匹配，此时并不报错
127.0.0.1:6379> LPOP a:stock
QUEUED
#发送事务中的第二个操作
127.0.0.1:6379> DECR b:stock
QUEUED
#实际执行事务，事务第一个操作执行报错
127.0.0.1:6379> EXEC
1) (error) WRONGTYPE Operation against a key holding the wrong kind of value
2) (integer) 8
```

因此，Redis 对事务原子性属性的保证情况：

- Redis 事务正常执行，可以保证原子性；
- Redis 事务执行中某一个操作执行失败，不保证原子性；

## # 日志

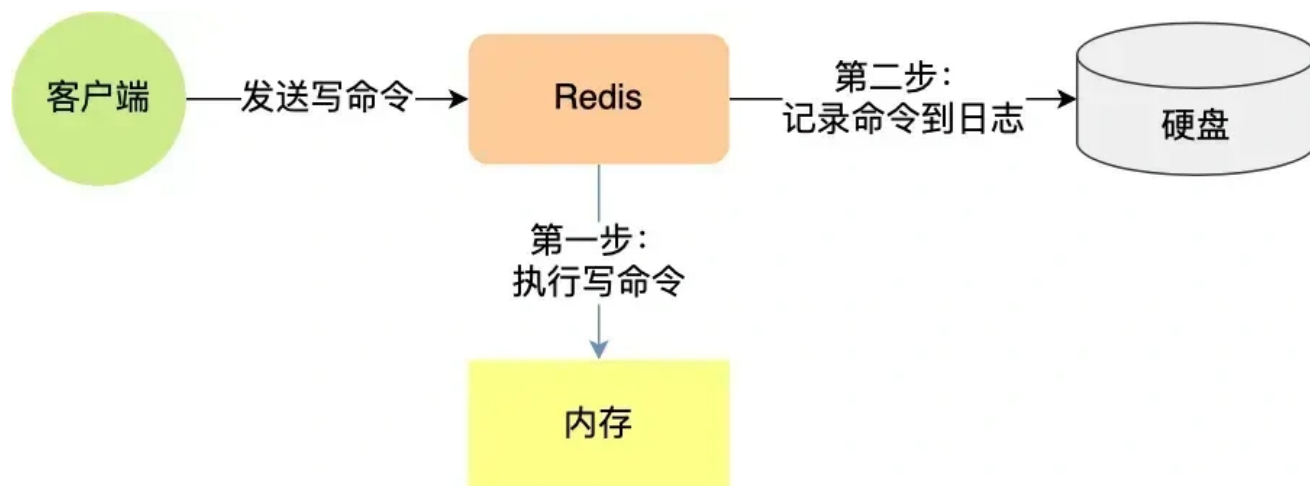
### # Redis有哪2种持久化方式？ 各自的优缺点是什么？

Redis 的读写操作都是在内存中，所以 Redis 性能才会高，但是当 Redis 重启后，内存中的数据就会丢失，那为了保证内存中的数据不会丢失，Redis 实现了数据持久化的机制，这个机制会把数据存储到磁盘，这样在 Redis 重启就能够从磁盘中恢复原有的数据。Redis 共有三种数据持久化的方式：

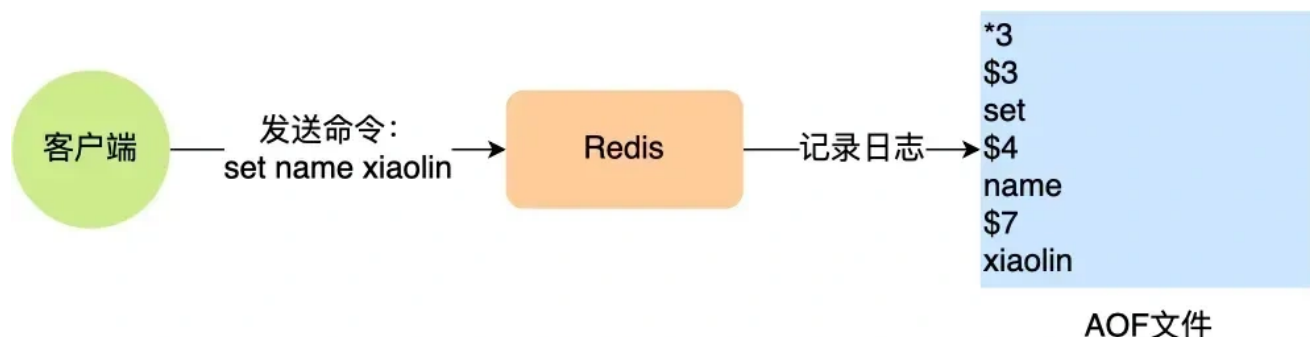
- **AOF 日志**：每执行一条写操作命令，就把该命令以追加的方式写入到一个文件里；
- **RDB 快照**：将某一时刻的内存数据，以二进制的方式写入磁盘；

## AOF 日志是如何实现的？

Redis 在执行完一条写操作命令后，就会把该命令以追加的方式写入到一个文件里，然后 Redis 重启时，会读取该文件记录的命令，然后逐一执行命令的方式来进行数据恢复。



我这里以「`set name xiaolin`」命令作为例子，Redis 执行了这条命令后，记录在 AOF 日志里的内容如下图：



Redis 提供了 3 种写回硬盘的策略，在 `Redis.conf` 配置文件中的 `appendfsync` 配置项可以有以下 3 种参数可填：

- **Always**，这个单词的意思是「总是」，所以它的意思是每次写操作命令执行完后，同步将 AOF 日志数据写回硬盘；
- **Everysec**，这个单词的意思是「每秒」，所以它的意思是每次写操作命令执行完后，先将命令写入到 AOF 文件的内核缓冲区，然后每隔一秒将缓冲区里的内容写回到硬盘；
- **No**，意味着不由 Redis 控制写回硬盘的时机，转交给操作系统控制写回的时机，也就是每次写操作命令执行完后，先将命令写入到 AOF 文件的内核缓冲区，再由操作系统决定何时将缓冲区内容写回硬盘。

我也把这 3 个写回策略的优缺点总结成了一张表格：

| 写回策略     | 写回时机      | 优点              | 缺点                |
|----------|-----------|-----------------|-------------------|
| Always   | 同步写回      | 可靠性高、最大程度保证数不丢失 | 每个写命令都要写回硬盘，性能开销大 |
| Everysec | 每秒写回      | 性能适中            | 宕机时会丢失1秒内的数据      |
| No       | 由操作系统控制写回 | 性能好             | 宕机时丢失的数据可能会很多     |

RDB 快照是如何实现的呢？

因为 AOF 日志记录的是操作命令，不是实际的数据，所以用 AOF 方法做故障恢复时，**需要全量把日志都执行一遍，一旦 AOF 日志非常多，势必会造成 Redis 的恢复操作缓慢**。为了解决这个问题，Redis 增加了 RDB 快照。

所谓的快照，就是记录某一个瞬间东西，比如当我们给风景拍照时，那一个瞬间的画面和信息就记录到了一张照片。所以，RDB 快照就是记录某一个瞬间的内存数据，记录的是实际数据，而 AOF 文件记录的是命令操作的日志，而不是实际的数据。因此在 Redis 恢复数据时，RDB 恢复数据的效率会比 AOF 高些，因为直接将 RDB 文件读入内存就可以，不需要像 AOF 那样还需要额外执行操作命令的步骤才能恢复数据。

Redis 提供了两个命令来生成 RDB 文件，分别是 save 和 bgsave，他们的区别就在于是否在「主线程」里执行：

- 执行了 save 命令，就会在主线程生成 RDB 文件，由于和执行操作命令在同一个线程，所以如果写入 RDB 文件的时间太长，**会阻塞主线程**；
- 执行了 bgsave 命令，会创建一个子进程来生成 RDB 文件，这样可以**避免主线程的阻塞**；

AOF和RDB优缺点

**AOF：**

- **优点：**首先，AOF提供了更好的数据安全性，因为它默认每接收到一个写命令就会追加到文件末尾。即使 Redis 服务器宕机，也只会丢失最后一次写入前的数据。其次，AOF支持多种同步策略（如everysec、always 等），可以根据需要调整数据安全性和性能之间的平衡。同时，AOF文件在Redis启动时可以通过重写机制优化，减少文件体积，加快恢复速度。并且，即使文件发生损坏，AOF还提供了redis-check-aof工具来修复损坏的文件。
- **缺点：**因为记录了每一个写操作，所以AOF文件通常比RDB文件更大，消耗更多的磁盘空间。并且，频繁的磁盘IO操作（尤其是同步策略设置为always时）可能会对Redis的写入性能造成一定影响。而且，当文件体积过大时，AOF会进行重写操作，AOF如果没有开启AOF重写或者重写频率较低，恢复过程可能较慢，因为它需要重放所有的操作命令。

**RDB：**

- **优点：**RDB通过快照的形式保存某一时刻的数据状态，文件体积小，备份和恢复的速度非常快。并且，RDB是在主线程之外通过fork子进程来进行的，不会阻塞服务器处理命令请求，对Redis服务的性能影响较小。最后，由于是定期快照，RDB文件通常比AOF文件小得多。
- **缺点：**RDB方式在两次快照之间，如果Redis服务器发生故障，这段时间的数据将会丢失。并且，如果在RDB创建快照到恢复期间有写操作，恢复后的数据可能与故障前的数据不完全一致。

# # 缓存淘汰和过期删除

## # 过期删除策略和内存淘汰策略有什么区别？

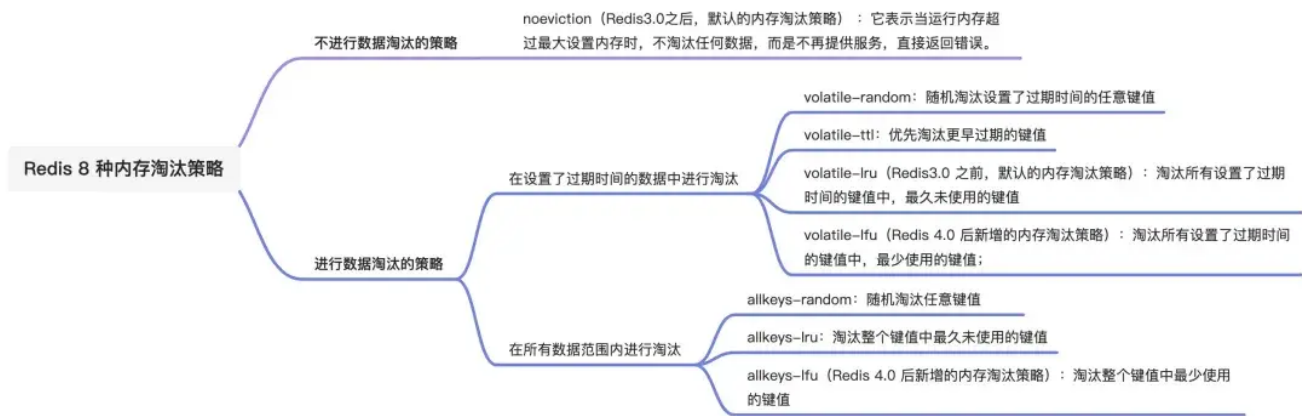
区别：

- 内存淘汰策略是在内存满了的时候，redis 会触发内存淘汰策略，来淘汰一些不必要的内存资源，以腾出空间，来保存新的内容
- 过期键删除策略是将已过期的键值对进行删除，Redis 采用的删除策略是惰性删除+定期删除。

## # 介绍一下Redis 内存淘汰策略

在 32 位操作系统中，maxmemory 的默认值是 3G，因为 32 位的机器最大只支持 4GB 的内存，而系统本身就需要一定的内存资源来支持运行，所以 32 位操作系统限制最大 3 GB 的可用内存是非常合理的，这样可以避免因为内存不足而导致 Redis 实例崩溃。

Redis 内存淘汰策略共有八种，这八种策略大体分为「不进行数据淘汰」和「进行数据淘汰」两类策略。



### 1、不进行数据淘汰的策略：

- **noeviction** (Redis3.0之后，默认的内存淘汰策略)：它表示当运行内存超过最大设置内存时，不淘汰任何数据，这时如果有新的数据写入，会报错通知禁止写入，不淘汰任何数据，但是如果没用数据写入的话，只是单纯的查询或者删除操作的话，还是可以正常工作。

### 2、进行数据淘汰的策略：

针对「进行数据淘汰」这一类策略，又可以细分为「在设置了过期时间的数据中进行淘汰」和「在所有数据范围内进行淘汰」这两类策略。

- 在设置了过期时间的数据中进行淘汰：
- **volatile-random**：随机淘汰设置了过期时间的任意键值；
- **volatile-ttl**：优先淘汰更早过期的键值。
- **volatile-lru** (Redis3.0 之前，默认的内存淘汰策略)：淘汰所有设置了过期时间的键值中，最久未使用的键值；
- **volatile-lfu** (Redis 4.0 后新增的内存淘汰策略)：淘汰所有设置了过期时间的键值中，最少使用的键值；
- 在所有数据范围内进行淘汰：
- **allkeys-random**：随机淘汰任意键值；
- **allkeys-lru**：淘汰整个键值中最久未使用的键值；
- **allkeys-lfu** (Redis 4.0 后新增的内存淘汰策略)：淘汰整个键值中最少使用的键值。

## # 介绍一下Redis过期删除策略

**Redis 选择「惰性删除+定期删除」这两种策略配和使用**，以求在合理使用 CPU 时间和避免内存浪费之间取得平衡。

Redis 的**惰性删除策略**由 db.c 文件中的 expireIfNeeded 函数实现，代码如下：

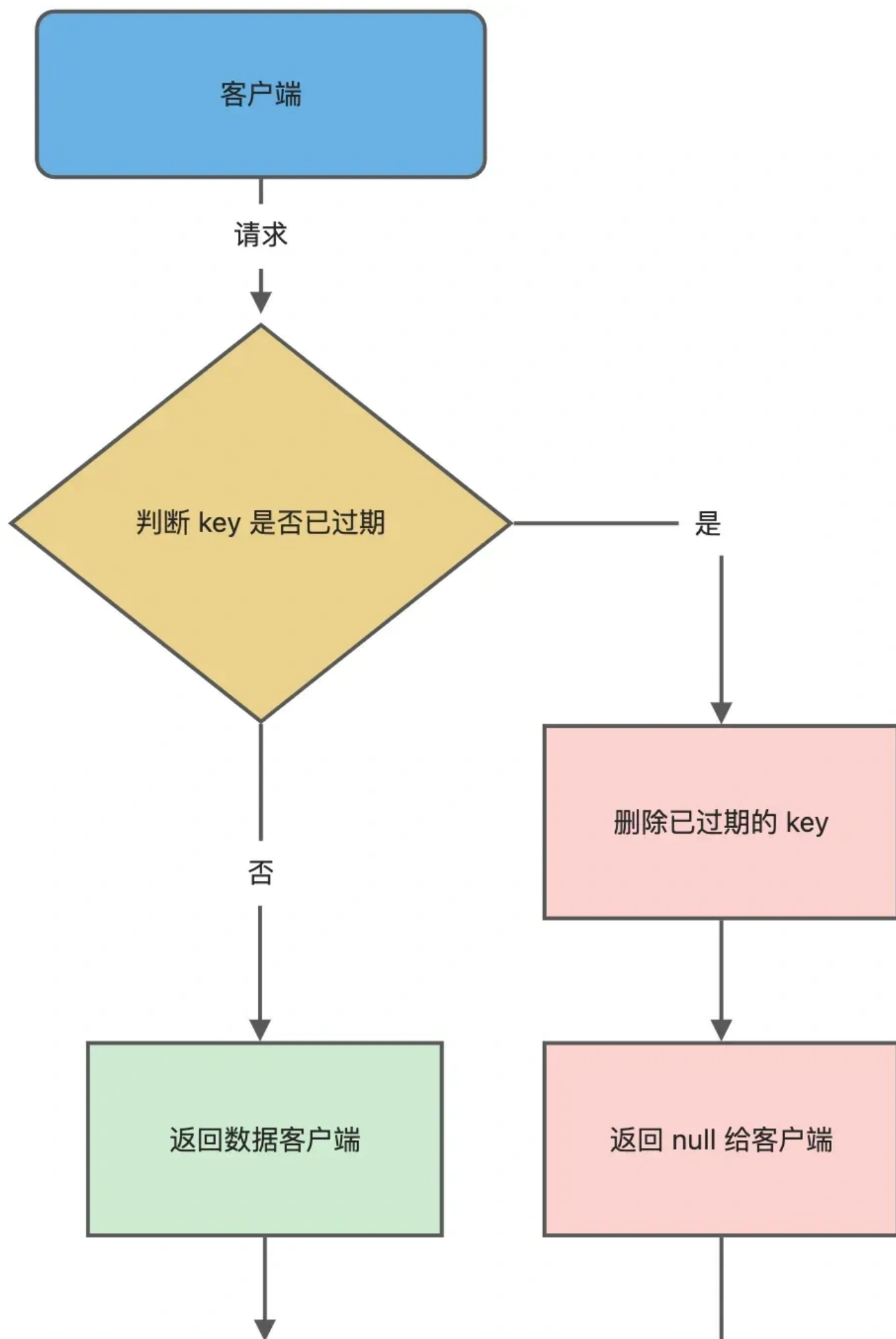
```
int expireIfNeeded(redisDb *db, robj *key) {
    // 判断 key 是否过期
    if (!keyIsExpired(db,key)) return 0;
    ....
    /* 删除过期键 */
    ....
    // 如果 server.lazyfree_lazy_expire 为 1 表示异步删除，反之同步删除；
    return server.lazyfree_lazy_expire ? dbAsyncDelete(db,key) :
                                         dbSyncDelete(db,key);
}
```

Redis 在访问或者修改 key 之前，都会调用 expireIfNeeded 函数对其进行检查，检查 key 是否过期：

- 如果过期，则删除该 key，至于选择异步删除，还是选择同步删除，根据 lazyfree\_lazy\_expire 参数配置决定（Redis 4.0版本开始提供参数），然后返回 null 客户端；
- 如果没有过期，不做任何处理，然后返回正常的键值对给客户端；

惰性删除的流程图如下：







Redis 的定期删除是每隔一段时间「随机」从数据库中取出一定数量的 key 进行检查，并删除其中的过期key。

#### 1、这个间隔检查的时间是多长呢？

在 Redis 中，默认每秒进行 10 次过期检查一次数据库，此配置可通过 Redis 的配置文件 redis.conf 进行配置，配置键为 hz 它的默认值是 hz 10。特别强调下，每次检查数据库并不是遍历过期字典中的所有 key，而是从数据库中随机抽取一定数量的 key 进行过期检查。

#### 2、随机抽查的数量是多少呢？

我查了下源码，定期删除的实现在 expire.c 文件下的 activeExpireCycle 函数中，其中随机抽查的数量由 ACTIVE\_EXPIRE\_CYCLE\_LOOKUPS\_PER\_LOOP 定义的，它是写在代码中的，数值是 20。也就是说，数据库每轮抽查时，会随机选择 20 个 key 判断是否过期。接下来，详细说说 Redis 的定期删除的流程：

1. 从过期字典中随机抽取 20 个 key；
2. 检查这 20 个 key 是否过期，并删除已过期的 key；
3. 如果本轮检查的已过期 key 的数量，超过 5 个 (20/4)，也就是「已过期 key 的数量」占比「随机抽取 key 的数量」大于 25%，则继续重复步骤 1；如果已过期的 key 比例小于 25%，则停止继续删除过期 key，然后等待下一轮再检查。

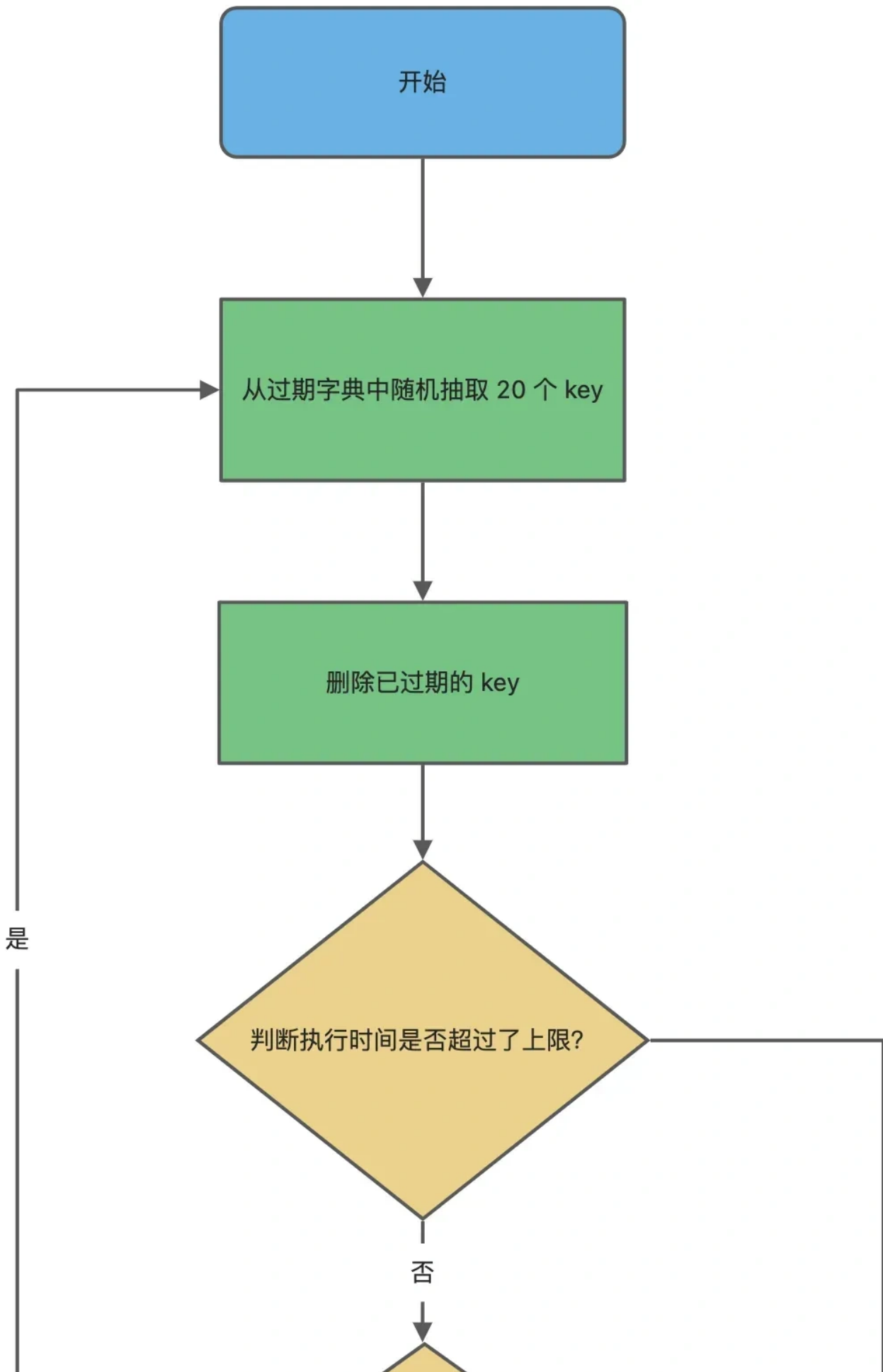
可以看到，定期删除是一个循环的流程。那 Redis 为了保证定期删除不会出现循环过度，导致线程卡死现象，为此增加了定期删除循环流程的时间上限，默认不会超过 25ms。针对定期删除的流程，我写了个伪代码：

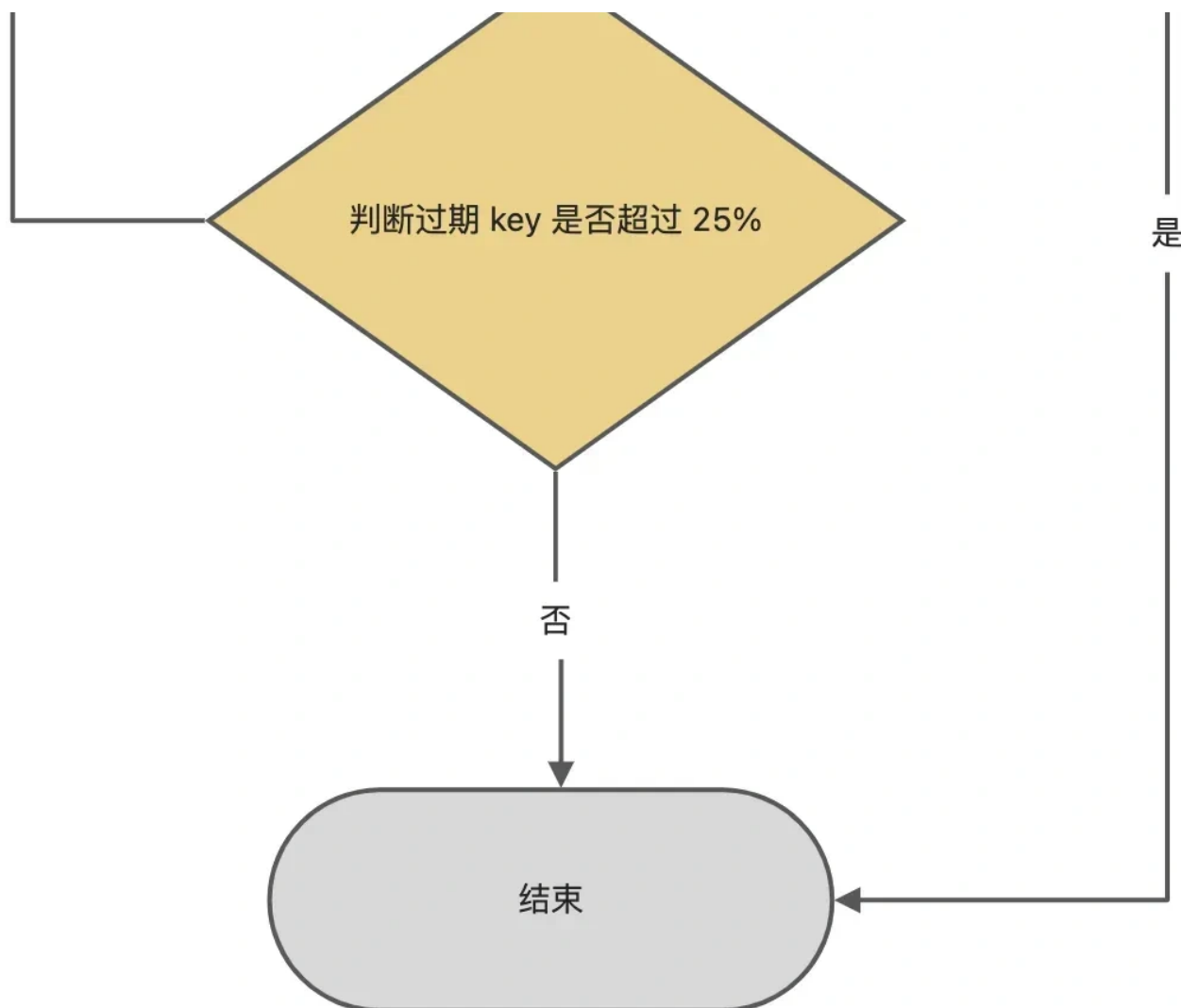
```
do {
    //已过期的数量
    expired = 0;
    //随机抽取的数量
    num = 20;
    while (num-->0) {
        //1. 从过期字典中随机抽取 1 个 key
        //2. 判断该 key 是否过期，如果已过期则进行删除，同时对 expired++
    }

    // 超过时间限制则退出
    if (timelimit_exit) return;

    /* 如果本轮检查的已过期 key 的数量，超过 25%，则继续随机抽查，否则退出本轮检查 */
} while (expired > 20/4);
```

定期删除的流程如下：





## # Redis的缓存失效会不会立即删除？

不会，Redis 的过期删除策略是选择「**惰性删除+定期删除**」这两种策略配和使用。

- 惰性删除策略的做法是，**不主动删除过期键**，每次从数据库访问 key 时，都检测 key 是否过期，如果过期则删除该 key。
- 定期删除策略的做法是，每隔一段时间「随机」从数据库中取出一定数量的 key 进行检查，并删除其中的过期key。

## # 那为什么我不过期立即删除？

在过期 key 比较多的情况下，删除过期 key 可能会占用相当一部分 CPU 时间，在内存不紧张但 CPU 时间紧张的情况下，将 CPU 时间用于删除和当前任务无关的过期键上，无疑会对服务器的响应时间和吞吐量造成影响。所以，定时删除策略对 CPU 不友好。

## # 集群

## # Redis主从同步中的增量和完全同步怎么实现？

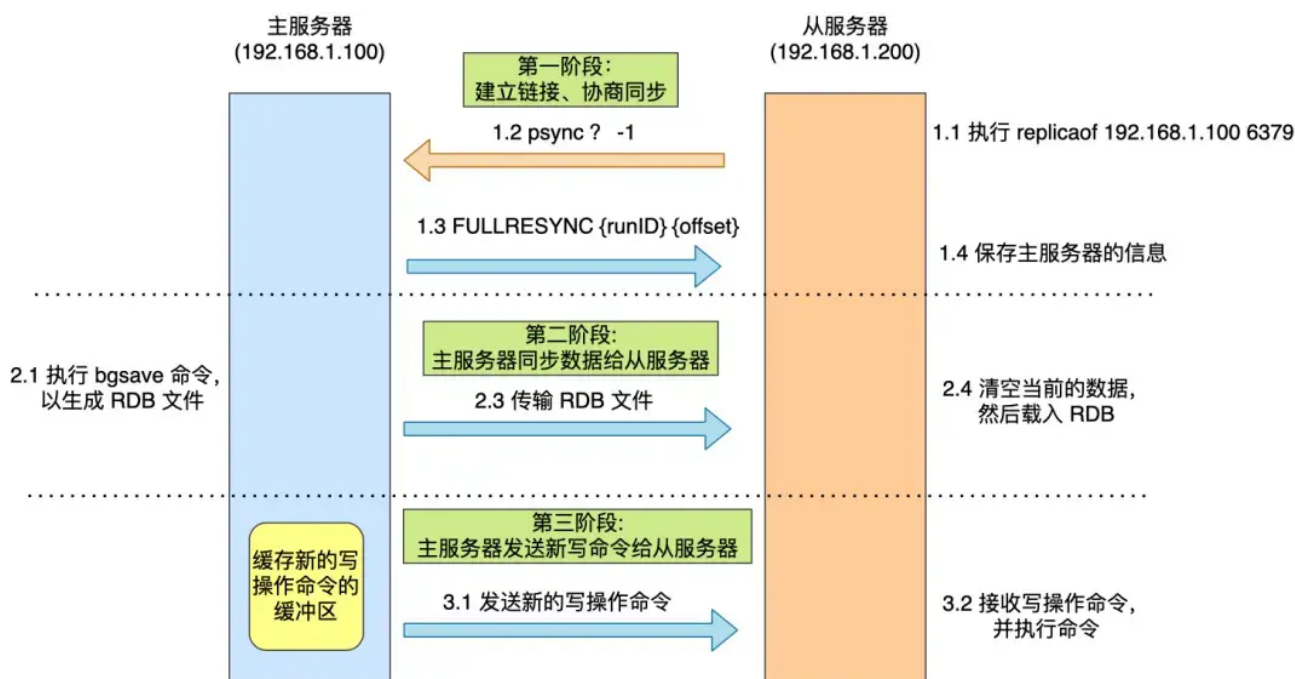
完全同步

完全同步发生在以下几种情况：

- **初次同步**：当一个从服务器（slave）首次连接到主服务器（master）时，会进行一次完全同步。
- **从服务器数据丢失**：如果从服务器数据由于某种原因（如断电）丢失，它会请求进行完全同步。
- **主服务器数据发生变化**：如果从服务器长时间未与主服务器同步，导致数据差异太大，也可能触发完全同步。

主从服务器间的第一次同步的过程可分为三个阶段：

- 第一阶段是建立链接、协商同步；
- 第二阶段是主服务器同步数据给从服务器；
- 第三阶段是主服务器发送新写操作命令给从服务器。

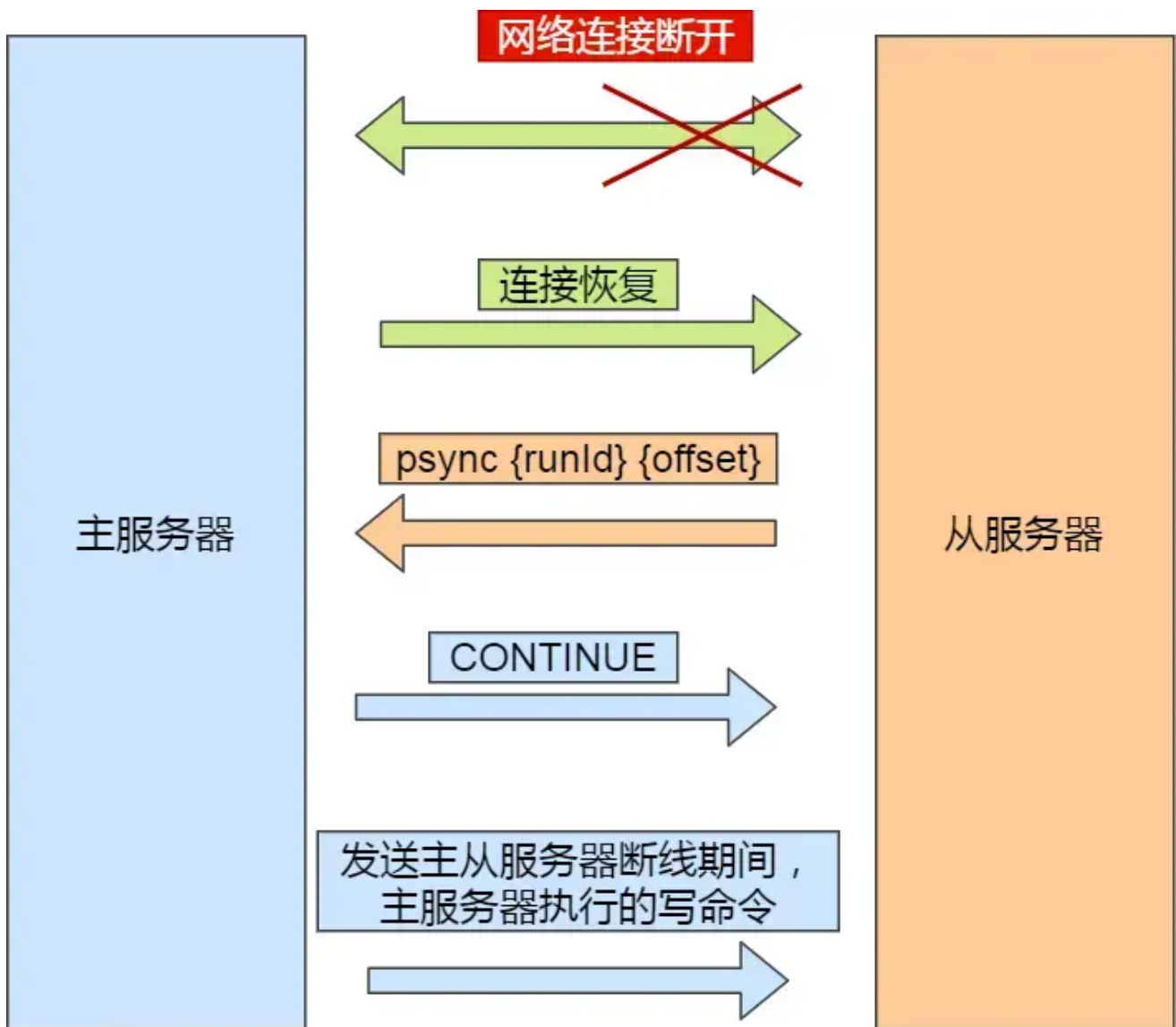


实现过程：

1. **从服务器发送SYNC命令**：从服务器向主服务器发送 `SYNC` 命令，请求开始同步。
2. **主服务器生成RDB快照**：接收到 `SYNC` 命令后，主服务器会保存当前数据集的状态到一个临时文件，这个过程称为RDB（Redis Database）快照。
3. **传输RDB文件**：主服务器将生成的RDB文件发送给从服务器。
4. **从服务器接收并应用RDB文件**：从服务器接收RDB文件后，会清空当前的数据集，并载入RDB文件中的数据。
5. **主服务器记录写命令**：在RDB文件生成和传输期间，主服务器会记录所有接收到的写命令到 `replication backlog buffer`。
6. **传输写命令**：一旦RDB文件传输完成，主服务器会将 `replication backlog buffer` 中的命令发送给从服务器，从服务器会执行这些命令，以保证数据的一致性。

#### 增量同步

增量同步允许从服务器从断点处继续同步，而不是每次都进行完全同步。它基于 `PSYNC` 命令，使用了运行ID（run ID）和复制偏移量（offset）的概念。



主要有三个步骤：

- 从服务器在恢复网络后，会发送 psync 命令给主服务器，此时的 psync 命令里的 offset 参数不是 -1；
- 主服务器收到该命令后，然后用 CONTINUE 响应命令告诉从服务器接下来采用增量复制的方式同步数据；
- 然后主服务器将主从服务器断线期间，所执行的写命令发送给从服务器，然后从服务器执行这些命令。

那么关键的问题来了，**主服务器怎么知道要将哪些增量数据发送给从服务器呢？**

答案藏在这两个东西里：

- **repl\_backlog\_buffer**，是一个「环形」缓冲区，用于主从服务器断连后，从中找到差异的数据；
- **replication offset**，标记上面那个缓冲区的同步进度，主从服务器都有各自的偏移量，主服务器使用 master\_repl\_offset 来记录自己「写」到的位置，从服务器使用 slave\_repl\_offset 来记录自己「读」到的位置。

那 repl\_backlog\_buffer 缓冲区是什么时候写入的呢？

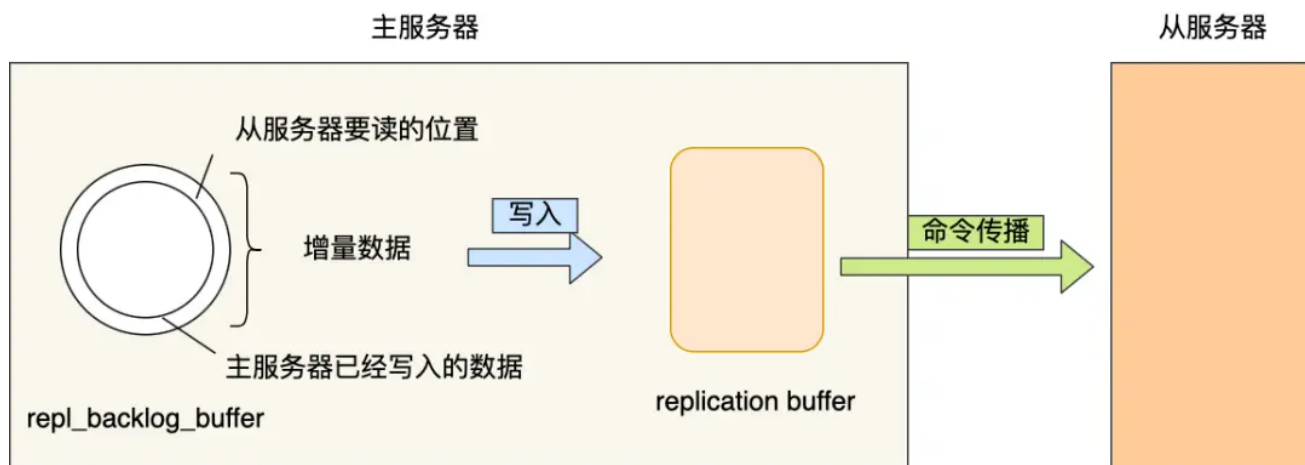
在主服务器进行命令传播时，不仅会将写命令发送给从服务器，还会将写命令写入到 repl\_backlog\_buffer 缓冲区里，因此这个缓冲区里会保存着最近传播的写命令。



网络断开后，当从服务器重新连上主服务器时，从服务器会通过 `psync` 命令将自己的复制偏移量 `slave_repl_offset` 发送给主服务器，主服务器根据自己的 `master_repl_offset` 和 `slave_repl_offset` 之间的差距，然后来决定对从服务器执行哪种同步操作：

- 如果判断出从服务器要读取的数据还在 `repl_backlog_buffer` 缓冲区里，那么主服务器将采用**增量同步**的方式；
- 相反，如果判断出从服务器要读取的数据已经不存在 `repl_backlog_buffer` 缓冲区里，那么主服务器将采用**全量同步**的方式。

当主服务器在 `repl_backlog_buffer` 中找到主从服务器差异（增量）的数据后，就会将增量的数据写入到 `replication buffer` 缓冲区，这个缓冲区我们前面也提到过，它是缓存将要传播给从服务器的命令。



`repl_backlog_buffer` 缓冲缓冲区的默认大小是 1M，并且由于它是一个环形缓冲区，所以当缓冲区写满后，主服务器继续写入的话，就会覆盖之前的数据。因此，当主服务器的写入速度远超于从服务器的读取速度，缓冲区的数据一下就会被覆盖。

那么在网络恢复时，如果从服务器想读的数据已经被覆盖了，主服务器就会采用全量同步，这个方式比增量同步的性能损耗要大很多。

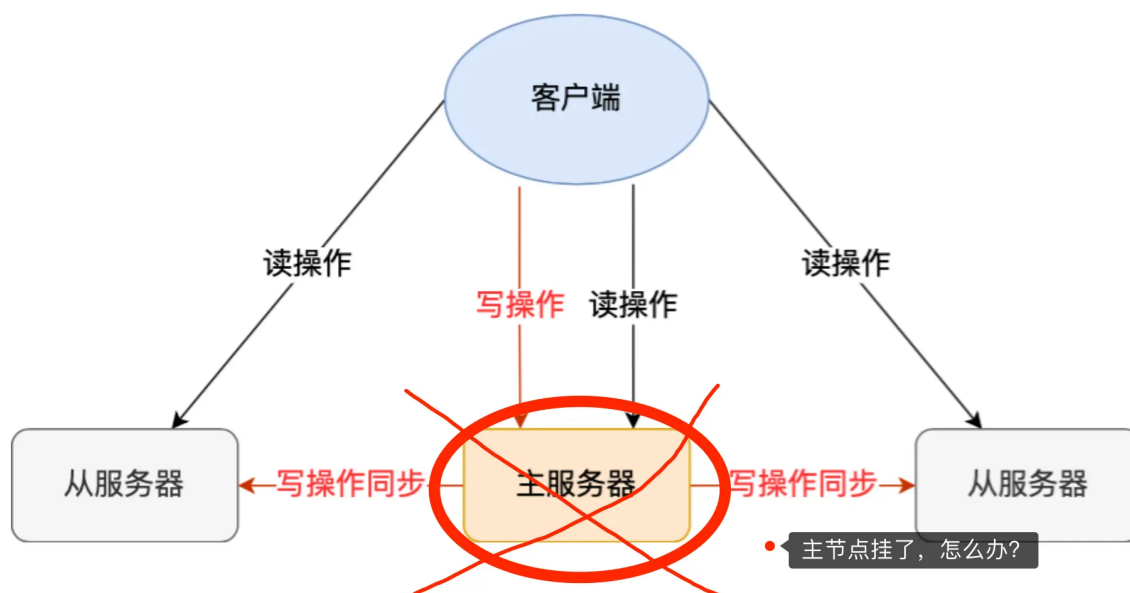
因此，为了避免在网络恢复时，主服务器频繁地使用全量同步的方式，我们应该调整下 `repl_backlog_buffer` 缓冲区大小，尽可能的大一些，减少出现从服务器要读取的数据被覆盖的概率，从而使得主服务器采用增量同步的方式。

## # redis主从和集群可以保证数据一致性吗？

redis 主从和集群在CAP理论都属于AP模型，即在面临网络分区时选择保证可用性和分区容忍性，而牺牲了强一致性。这意味着在网络分区的情况下，Redis主从复制和集群可以继续提供服务并保持可用，但可能会出现部分节点之间的数据不一致。

## # 哨兵机制原理是什么？

在 Redis 的主从架构中，由于主从模式是读写分离的，如果主节点（master）挂了，那么将没有主节点来服务客户端的写操作请求，也没有主节点给从节点（slave）进行数据同步了。



这时如果要恢复服务的话，需要人工介入，选择一个「从节点」切换为「主节点」，然后让其他从节点指向新的主节点，同时还需要通知上游那些连接 Redis 主节点的客户端，将其配置中的主节点 IP 地址更新为「新主节点」的 IP 地址。

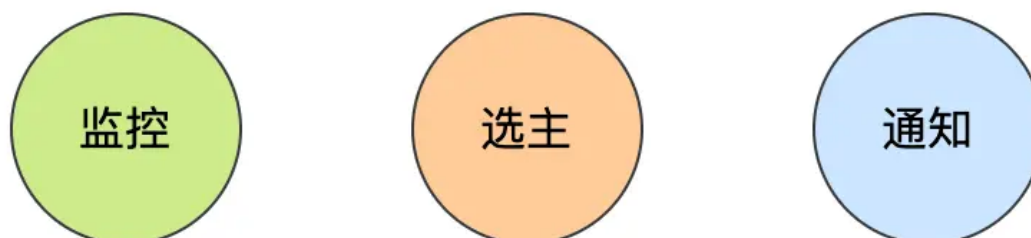
这样也不太“智能”了，要是有一个节点能监控「主节点」的状态，当发现主节点挂了，它自动将一个「从节点」切换为「主节点」的话，那么可以节省我们很多事情啊！

Redis 在 2.8 版本以后提供的**哨兵 (\*\*\*Sentinel\*\*\*) 机制**，它的作用是实现**主从节点故障转移**。它会监测主节点是否存活，如果发现主节点挂了，它就会选举一个从节点切换为主节点，并且把新主节点的相关信息通知给从节点和客户端。

哨兵其实是一个运行在特殊模式下的 Redis 进程，所以它也是一个节点。从“哨兵”这个名字也可以看得出来，它相当于是“观察者节点”，观察的对象是主从节点。

当然，它不仅仅是观察那么简单，在它观察到有异常的状况下，会做出一些“动作”，来修复异常状态。

哨兵节点主要负责三件事情：**监控、选主、通知**。



## # 哨兵机制的选主节点的算法介绍一下

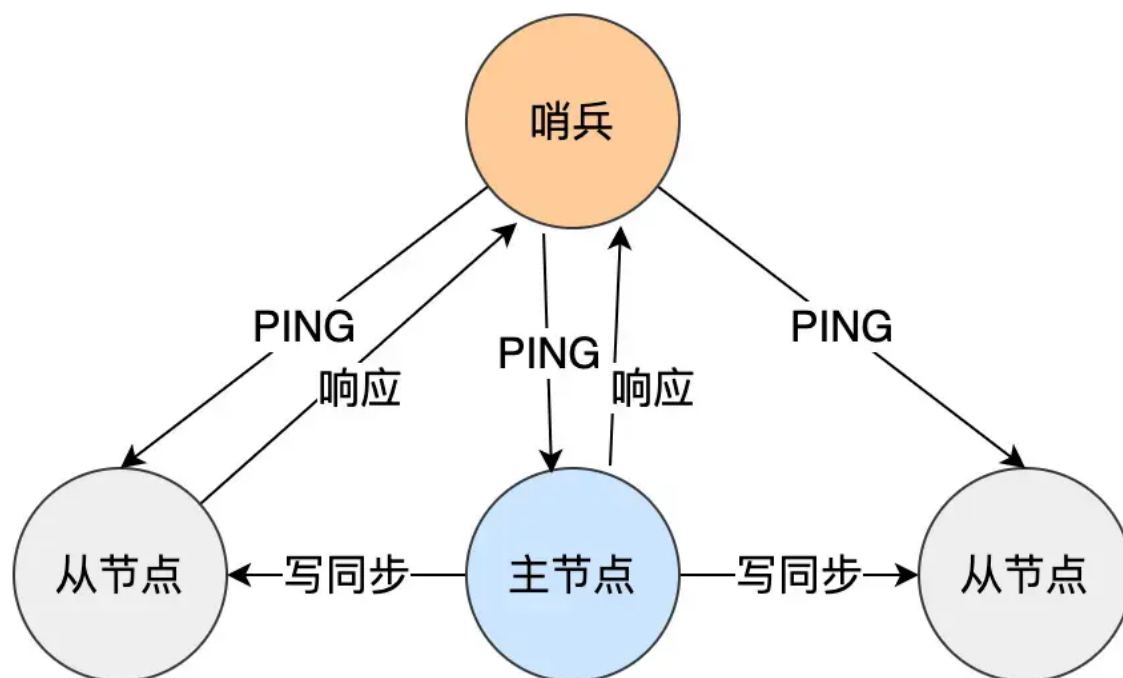
当redis集群的主节点故障时，Sentinel集群将从剩余的从节点中选举一个新的主节点，有以下步骤：

1. 故障节点主观下线

- 故障节点客观下线
- Sentinel集群选举Leader
- Sentinel Leader决定新主节点

#### 1. 故障节点主观下线

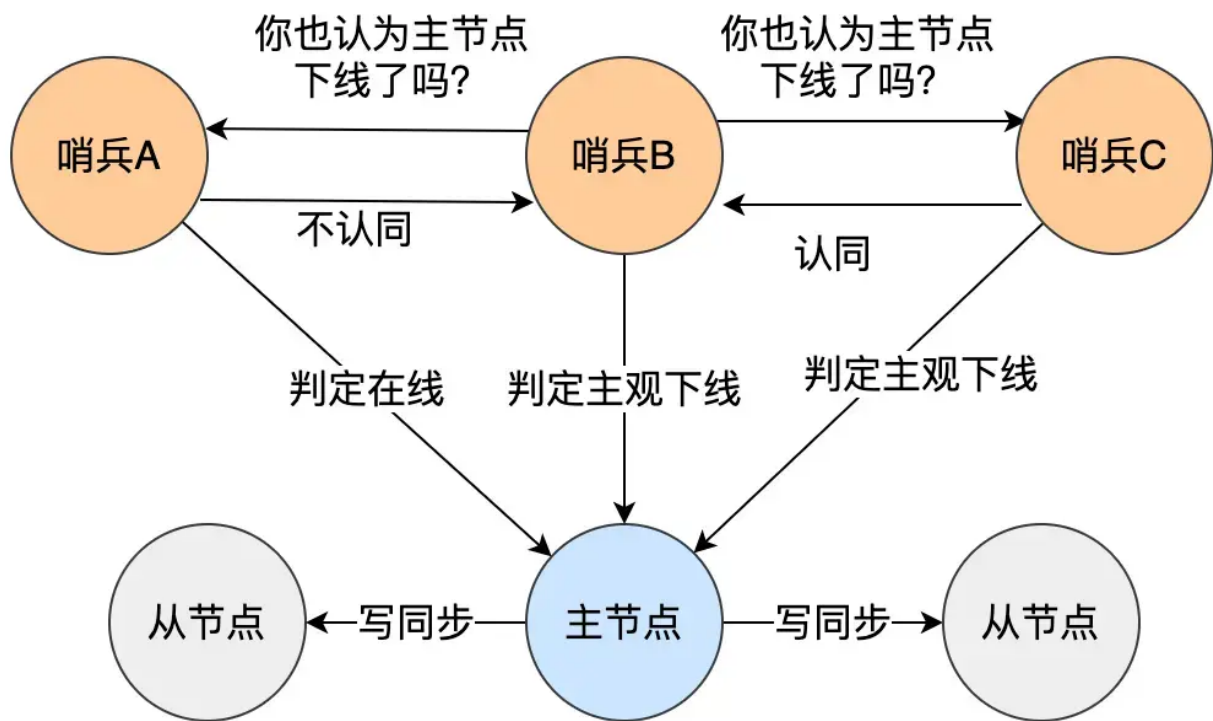
Sentinel集群的每一个Sentinel节点会定时对redis集群的所有节点发心跳包检测节点是否正常。如果一个节点在down-after-milliseconds时间内没有回复Sentinel节点的心跳包，则该redis节点被该Sentinel节点主观下线。



#### 1. 故障节点客观下线

当节点被一个Sentinel节点记为主观下线时，并不意味着该节点肯定故障了，还需要Sentinel集群的其他Sentinel节点共同判断为主观下线才行。

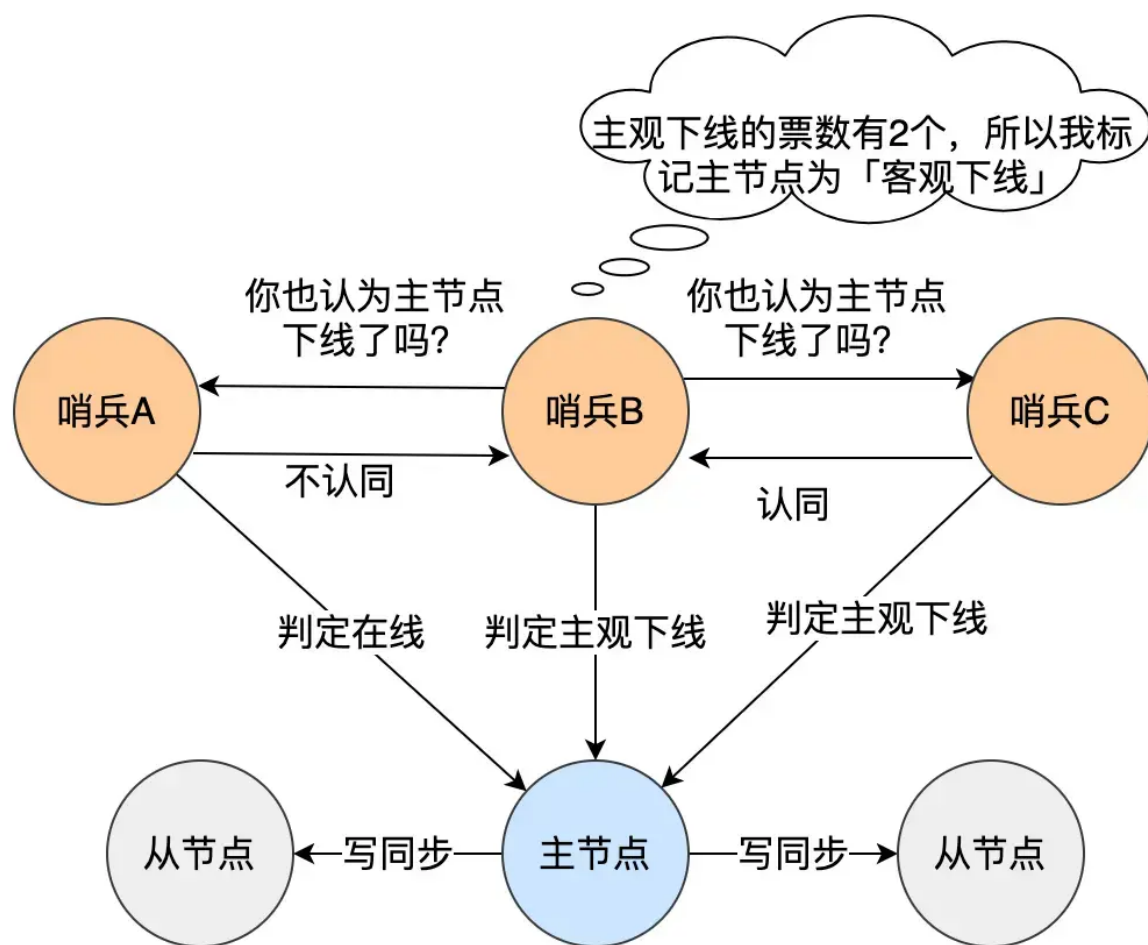
该Sentinel节点会询问其他Sentinel节点，如果Sentinel集群中超过quorum数量的Sentinel节点认为该redis节点主观下线，则该redis客观下线。



如果客观下线的redis节点是从节点或者是Sentinel节点，则操作到此为止，没有后续的操作了；如果客观下线的redis节点为主节点，则开始故障转移，从从节点中选举一个节点升级为主节点。

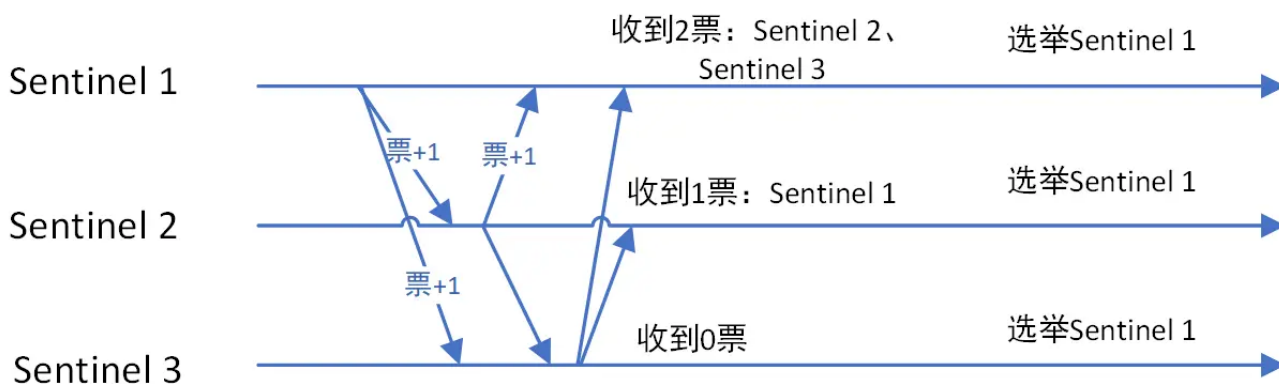
#### 1. Sentinel集群选举Leader

如果要从redis集群选举一个节点为主节点，首先需要从Sentinel集群中选举一个Sentinel节点作为Leader。



每一个Sentinel节点都可以成为Leader，当一个Sentinel节点确认redis集群的主节点主观下线后，会请求其他Sentinel节点要求将自己选举为Leader。被请求的Sentinel节点如果没有同意过其他Sentinel节点的选举请求，则同意该请求(选举票数+1)，否则不同意。

如果一个Sentinel节点获得的选举票数达到Leader最低票数(quorum和Sentinel节点数/2+1的最大值)，则该Sentinel节点选举为Leader；否则重新进行选举。

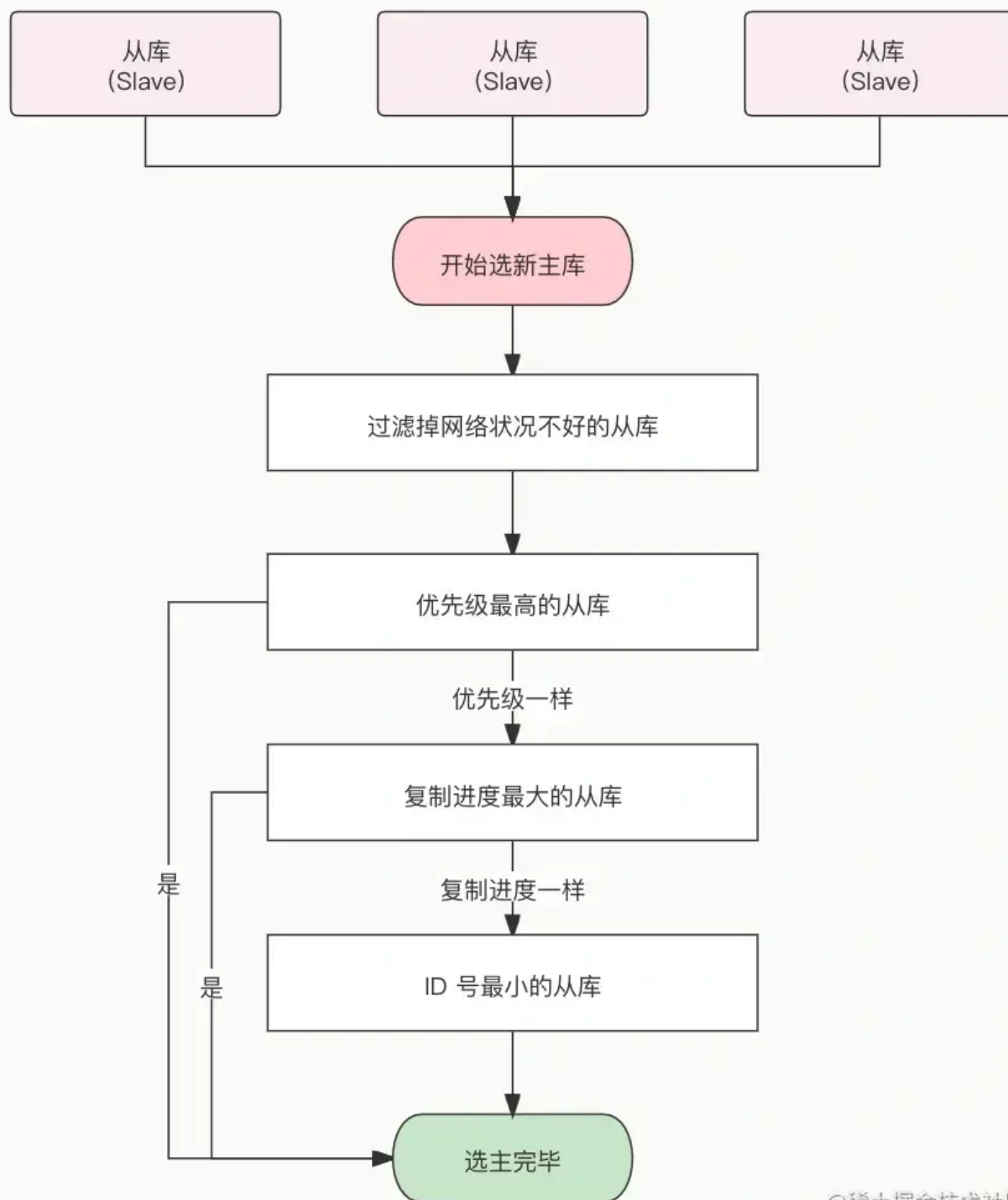


举个例子，假设哨兵节点有3个，quorum 设置为2，那么任何一个想成为 Leader 的哨兵只要拿到2张赞成票，就可以选举成功了。如果没有满足条件，就需要重新进行选举。

## 1. Sentinel Leader决定新主节点

当Sentinel集群选举出Sentinel Leader后，由Sentinel Leader从redis从节点中选择一个redis节点作为主节点：

1. 过滤故障的节点
2. 选择优先级slave-priority最大的从节点作为主节点，如不存在则继续
3. 选择复制偏移量（数据写入量的字节，记录写了多少数据。主服务器会把偏移量同步给从服务器，当主从的偏移量一致，则数据是完全同步）最大的从节点作为主节点，如不存在则继续
4. 选择runid（redis每次启动的时候生成随机的runid作为redis的标识）最小的从节点作为主节点





## # Redis集群的模式了解吗 优缺点了解吗

当 Redis 缓存数据量大到一台服务器无法缓存时，就需要使用 **Redis 切片集群**（Redis Cluster）方案，它将数据分布在不同的服务器上，以此来降低系统对单主节点的依赖，从而提高 Redis 服务的读写性能。

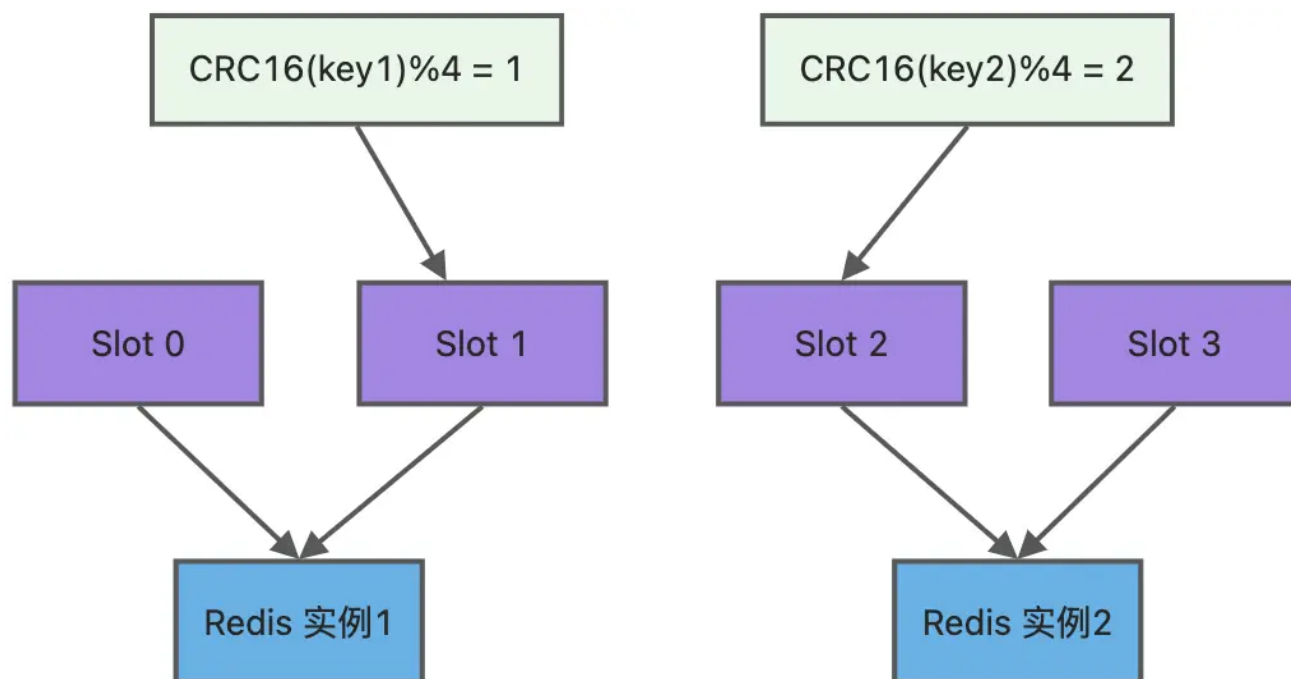
Redis Cluster 方案采用哈希槽（Hash Slot），来处理数据和节点之间的映射关系。在 Redis Cluster 方案中，**一个切片集群共有 16384 个哈希槽**，这些哈希槽类似于数据分区，每个键值对都会根据它的 key，被映射到一个哈希槽中，具体执行过程分为两大步：

- 根据键值对的 key，按照 CRC16 算法计算一个 16 bit 的值。
- 再用 16bit 值对 16384 取模，得到 0~16383 范围内的模数，每个模数代表一个相应编号的哈希槽。

接下来的问题就是，这些哈希槽怎么被映射到具体的 Redis 节点上的呢？有两种方案：

- **平均分配**：在使用 cluster create 命令创建 Redis 集群时，Redis 会自动把所有哈希槽平均分布到集群节点上。比如集群中有 9 个节点，则每个节点上槽的个数为  $16384/9$  个。
- **手动分配**：可以使用 cluster meet 命令手动建立节点间的连接，组成集群，再使用 cluster addslots 命令，指定每个节点上的哈希槽个数。

为了方便你的理解，我通过一张图来解释数据、哈希槽，以及节点三者的映射分布关系。



上图中的切片集群一共有 2 个节点，假设有 4 个哈希槽（Slot 0 ~ Slot 3）时，我们就可以通过命令手动分配哈希槽，比如节点 1 保存哈希槽 0 和 1，节点 2 保存哈希槽 2 和 3。

```
redis-cli -h 192.168.1.10 -p 6379 cluster addslots 0,1
redis-cli -h 192.168.1.11 -p 6379 cluster addslots 2,3
```

然后在集群运行的过程中，key1 和 key2 计算完 CRC16 值后，对哈希槽总个数 4 进行取模，再根据各自的模数结果，就可以被映射到哈希槽 1（对应节点1）和 哈希槽 2（对应节点2）。

需要注意的是，在手动分配哈希槽时，需要把 16384 个槽都分配完，否则 Redis 集群无法正常工作。

优点：

- **高可用性**：Redis集群最主要的优点是提供了高可用性，节点之间采用主从复制机制，可以保证数据的持久性和容错能力，哪怕其中一个节点挂掉，整个集群还可以继续工作。
- **高性能**：Redis集群采用分片技术，将数据分散到多个节点，从而提高读写性能。当业务访问量大到单机Redis无法满足时，可以通过添加节点来增加集群的吞吐量。
- **扩展性好**：Redis集群的扩展性非常好，可以根据实际需求动态增加或减少节点，从而实现可扩展性。集群模式中的某些节点还可以作为代理节点，自动转发请求，增加数据模式的灵活度和可定制性。

缺点：

- **部署和维护较复杂**：Redis集群的部署和维护需要考虑到分片规则、节点的布置、主从配置以及故障处理等多个方面，需要较强的技术支持，增加了节点异常处理的复杂性和成本。
- **集群同步问题**：当某些节点失败或者网络出故障，集群中数据同步的问题也会出现。数据同步的复杂度和工作量随着节点的增加而增加，同步时间也较长，导致一定的读写延迟。
- **数据分片限制**：Redis集群的数据分片也限制了一些功能的实现，如在一个key上修改多次，可能会因为该key所在的节点位置变化而失败。此外，由于将数据分散存储到各个节点，某些操作不能跨节点实现，不同节点之间的一些操作需要额外注意。

## # 场景

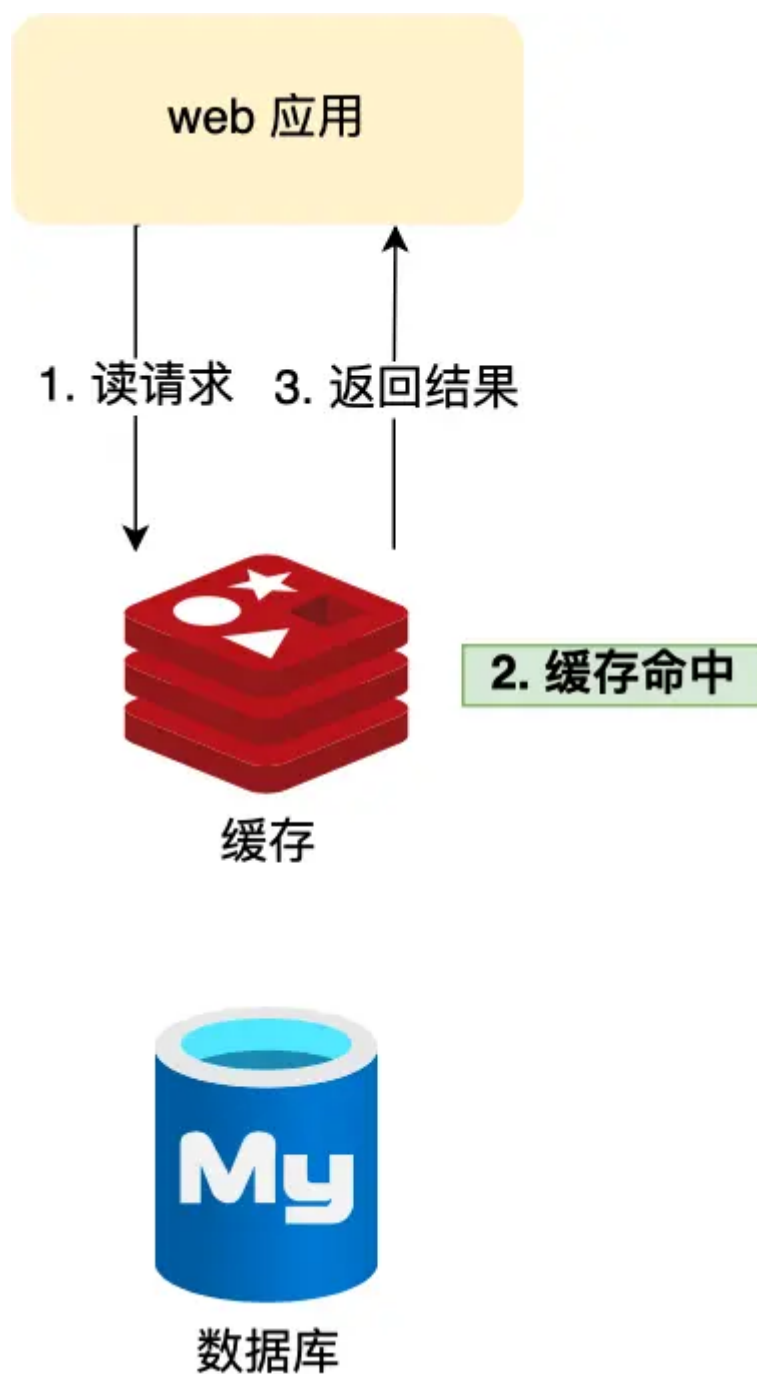
---

### # 为什么使用redis?

主要是因为 **Redis 具备「高性能」和「高并发」两种特性**。

#### 1、Redis 具备高性能

假如用户第一次访问 MySQL 中的某些数据。这个过程会比较慢，因为是从硬盘上读取的。将该用户访问的数据缓存在 Redis 中，这样下一次再访问这些数据的时候就可以直接从缓存中获取了，操作 Redis 缓存就是直接操作内存，所以速度相当快。



如果 MySQL 中的对应数据改变的之后，同步改变 Redis 缓存中相应的数据即可，不过这里会有 Redis 和 MySQL 双写一致性的问题。

## 2、Redis 具备高并发

单台设备的 Redis 的 QPS (Query Per Second, 每秒钟处理完请求的次数) 是 MySQL 的 10 倍, Redis 单机的 QPS 能轻松破 10w, 而 MySQL 单机的 QPS 很难破 1w。

所以，直接访问 Redis 能够承受的请求是远远大于直接访问 MySQL 的，所以我们可以考虑把数据库中的部分数据转移到缓存中去，这样用户的一部分请求会直接到缓存这里而不用经过数据库。

## # 为什么redis比mysql要快？

- **内存存储**：Redis 是基于内存存储的 NoSQL 数据库，而 MySQL 是基于磁盘存储的关系型数据库。由于内存存储速度快，Redis 能够更快地读取和写入数据，而无需像 MySQL 那样频繁进行磁盘 I/O 操作。

- **简单数据结构**：Redis 是基于键值对存储数据的，支持简单的数据结构（字符串、哈希、列表、集合、有序集合）。相比之下，MySQL 需要定义表结构、索引等复杂的关系型数据结构，因此在某些场景下 Redis 的数据操作更为简单高效，比如 Redis 用哈希表查询，只需要O1 时间复杂度，而MySQL引擎的底层实现是B+Tree，时间复杂度是O(logn)
- **线程模型**：Redis 采用单线程模型可以避免多线程之间的竞争，省去了多线程切换带来的时间和性能上的开销，而且也不会导致死锁问题。

## # 本地缓存和Redis缓存的区别？

**本地缓存**是指将数据存储在本地应用程序或服务器上，通常用于加速数据访问和提高响应速度。本地缓存通常使用内存作为存储介质，利用内存的高速读写特性来提高数据访问速度。

本地缓存的优势：

- **访问速度快**：由于本地缓存存储在本地内存中，因此访问速度非常快，能够满足频繁访问和即时响应的需求。
- **减轻网络压力**：本地缓存能够降低对远程服务器的访问次数，从而减轻网络压力，提高系统的可用性和稳定性。
- **低延迟**：由于本地缓存位于本地设备上，因此能够提供低延迟的访问速度，适用于对实时性要求较高的应用场景。

本地缓存的不足：

- **可扩展性有限**：本地缓存的可扩展性受到硬件资源的限制，无法支持大规模的数据存储和访问。

**分布式缓存（Redis）**是指将数据存储在多个分布式节点上，通过协同工作来提供高性能的数据访问服务。分布式缓存通常使用集群方式进行部署，利用多台服务器来分担数据存储和访问的压力。

分布式缓存的优势：

- **可扩展性强**：分布式缓存的节点可以动态扩展，能够支持大规模的数据存储和访问需求。
- **数据一致性高**：通过分布式一致性协议，分布式缓存能够保证数据在多个节点之间的一致性，减少数据不一致的问题。
- **易于维护**：分布式缓存通常采用自动化管理方式，能够降低维护成本 and 管理的复杂性。

分布式缓存的不足：

- **访问速度相对较慢**：相对于本地缓存，分布式缓存的访问速度相对较慢，因为数据需要从多个节点进行访问和协同。
- **网络开销大**：由于分布式缓存需要通过网络进行数据传输和协同操作，因此相对于本地缓存来说，网络开销较大。

在选择使用本地缓存还是分布式缓存时，我们需要根据具体的应用场景和需求进行权衡。以下是一些考虑因素：

- **数据大小**：如果数据量较小，且对实时性要求较高，本地缓存更适合；如果数据量较大，且需要支持大规模的并发访问，分布式缓存更具优势。
- **网络状况**：如果网络状况良好且稳定，分布式缓存能够更好地发挥其优势；如果网络状况较差或不稳定，本地缓存的访问速度和稳定性可能更有优势。
- **业务特点**：对于实时性要求较

## # 高并发场景，Redis单节点+MySQL单节点能有多大的并发量？

- 如果缓存命中的话，4 核心 8g 内存的配置，redis 可以支撑 10w 的 qps
- 如果缓存没有命中的话，4 核心 8g 内存的配置，mysql 只能支持 5000 左右的 qps

## # redis应用场景是什么？

Redis 是一种基于内存的数据库，对数据的读写操作都是在内存中完成，因此读写速度非常快，常用于缓存、消息队列、分布式锁等场景。

- **缓存**: Redis最常见的用途就是作为缓存系统。通过将热门数据存储在内存中，可以极大地提高访问速度，减轻数据库负载，这对于需要快速响应时间的应用程序非常重要。
- **排行榜**: Redis的有序集合结构非常适合用于实现排行榜和排名系统，可以方便地进行数据排序和排名。
- **分布式锁**: Redis的特性可以用来实现分布式锁，确保多个进程或服务之间的数据操作的原子性和一致性。
- **计数器**: 由于Redis的原子操作和高性能，它非常适合用于实现计数器和统计数据的存储，如网站访问量统计、点赞数统计等。
- **消息队列**: Redis的发布订阅功能使其成为一个轻量级的消息队列，它可以用来实现发布和订阅模式，以便实时处理消息。

## # Redis除了缓存，还有哪些应用？

### Redis实现消息队列

- **使用Pub/Sub模式**: Redis的Pub/Sub是一种基于发布/订阅的消息模式，任何客户端都可以订阅一个或多个频道，发布者可以向特定频道发送消息，所有订阅该频道的客户端都会收到此消息。该方式实现起来比较简单，发布者和订阅者完全解耦，支持模式匹配订阅。但是这种方式不支持消息持久化，消息发布后若无订阅者在线则会被丢弃；不保证消息的顺序和可靠性传输。
- **使用List结构**: 使用List的方式通常是使用 `LPUSH` 命令将消息推入一个列表，消费者使用 `BLPOP` 或 `BRPOP` 阻塞地从列表中取出消息（先进先出FIFO）。这种方式可以实现简单的任务队列。这种方式可以结合Redis的过期时间特性实现消息的TTL；通过Redis事务可以保证操作的原子性。但是需要客户端自己实现消息确认、重试等机制，相比专门的消息队列系统功能较弱。

### Redis实现分布式锁

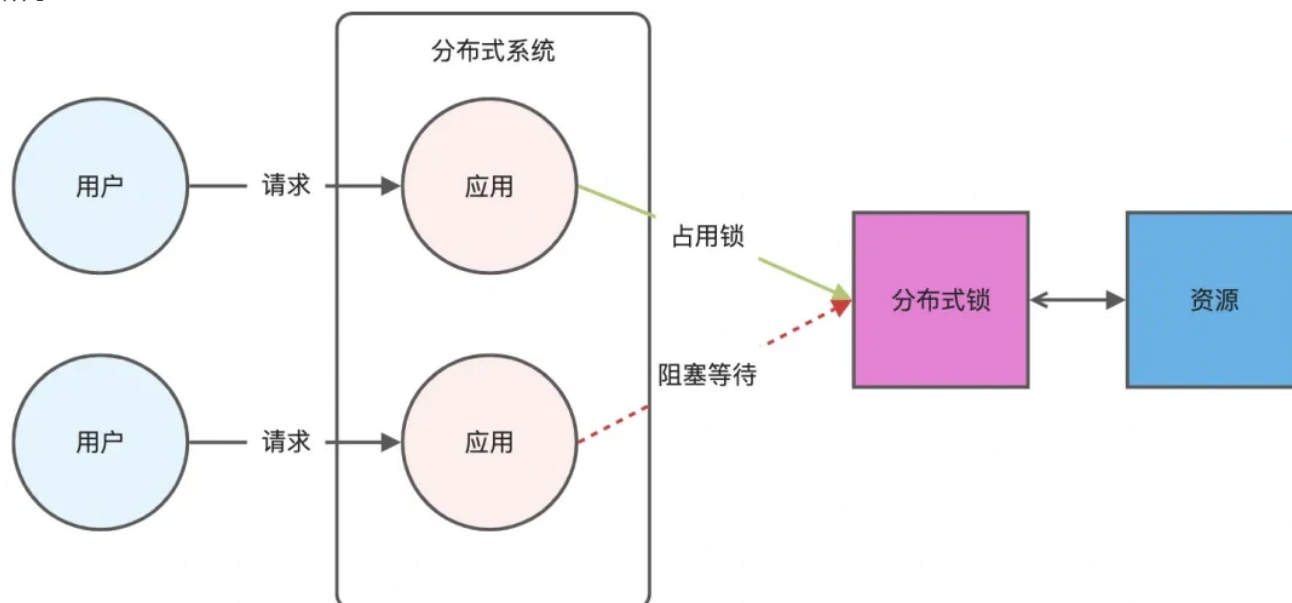
- **set nx方式**: Redis提供了几种方式来实现分布式锁，最常用的是基于 `SET` 命令的争抢锁机制。客户端可以使用 `SET resource_name lock_value NX PX milliseconds` 命令设置锁，其中 `NX` 表示只有当键不存在时才设置，`PX` 指定锁的有效时间（毫秒）。如果设置成功，则认为客户端获得锁。客户端完成操作后，解锁的还需要先判断锁是不是自己，再进行删除，这里涉及到 2 个操作，为了保证这两个操作的原子性，可以用 lua 脚本来实现。
- **RedLock算法**: 为了提高分布式锁的可靠性，Redis作者Antirez提出了RedLock算法，它基于多个独立的Redis实例来实现一个更安全的分布式锁。它的基本原理是客户端尝试在多数（大于半数）Redis实例上同时加锁，只有当在大多数实例上加锁成功时才认为获取锁成功。锁的超时时间应该远小于单个实例的超时时间，以避免死锁。该方式可以通过跨多个节点减少单点故障的影响，提高了锁的可用性和安全性。

## # Redis支持并发操作吗？

- **单个 Redis 命令的原子性**: Redis 的单个命令是原子性的，这意味着一个命令要么完全执行成功，要么完全不执行，确保操作的一致性。这对于并发操作非常重要。
- **多个操作的事务**: Redis 支持事务，可以将一系列的操作放在一个事务中执行，使用 `MULTI`、`EXEC`、`DISCARD` 和 `WATCH` 等命令来管理事务。这样可以确保一系列操作的原子性。

## # Redis分布式锁的实现原理？什么场景下用到分布式锁？

**分布式锁**是用于分布式环境下并发控制的一种机制，用于控制某个资源在同一时刻只能被一个应用所使用。如下图所示：



Redis 本身可以被多个客户端共享访问，正好就是一个共享存储系统，可以用来保存分布式锁，而且 Redis 的读写性能高，可以应对高并发的锁操作场景。Redis 的 SET 命令有个 NX 参数可以实现「key不存在才插入」，所以可以用它来实现分布式锁：

- 如果 key 不存在，则显示插入成功，可以用来表示加锁成功；
- 如果 key 存在，则会显示插入失败，可以用来表示加锁失败。

基于 Redis 节点实现分布式锁时，对于加锁操作，我们需要满足三个条件。

- 加锁包括了读取锁变量、检查锁变量值和设置锁变量值三个操作，但需要以原子操作的方式完成，所以，我们使用 SET 命令带上 NX 选项来实现加锁；
- 锁变量需要设置过期时间，以免客户端拿到锁后发生异常，导致锁一直无法释放，所以，我们在 SET 命令执行时加上 EX/PX 选项，设置其过期时间；
- 锁变量的值需要能区分来自不同客户端的加锁操作，以免在释放锁时，出现误释放操作，所以，我们使用 SET 命令设置锁变量值时，每个客户端设置的值是一个唯一值，用于标识客户端；

满足这三个条件的分布式命令如下：

```
SET lock_key unique_value NX PX 10000
```

- lock\_key 就是 key 键；
- unique\_value 是客户端生成的唯一的标识，区分来自不同客户端的锁操作；
- NX 代表只在 lock\_key 不存在时，才对 lock\_key 进行设置操作；
- PX 10000 表示设置 lock\_key 的过期时间为 10s，这是为了避免客户端发生异常而无法释放锁。

而解锁的过程就是将 lock\_key 键删除（del lock\_key），但不能乱删，要保证执行操作的客户端就是加锁的客户端。所以，解锁的时候，我们要先判断锁的 unique\_value 是否为加锁客户端，是的话，才将 lock\_key 键删除。

可以看到，解锁是有两个操作，这时就需要 Lua 脚本来保证解锁的原子性，因为 Redis 在执行 Lua 脚本时，可以以原子性的方式执行，保证了锁释放操作的原子性。



```
// 释放锁时，先比较 unique_value 是否相等，避免锁的误释放
if redis.call("get",KEYS[1]) == ARGV[1] then
    return redis.call("del",KEYS[1])
else
    return 0
end
```

这样一来，就通过使用 SET 命令和 Lua 脚本在 Redis 单节点上完成了分布式锁的加锁和解锁。

## # Redis的大Key问题是什么？

Redis大key问题指的是某个key对应的value值所占的内存空间比较大，导致Redis的性能下降、内存不足、数据不均衡以及主从同步延迟等问题。

到底多大的数据量才算是大key？

没有固定的判别标准，通常认为字符串类型的key对应的value值占用空间大于1M，或者集合类型的k元素数量超过1万个，就算是大key。

Redis大key问题的定义及评判准则并非一成不变，而应根据Redis的实际运用以及业务需求来综合评估。

例如，在高并发且低延迟的场景中，仅10kb可能就已构成大key；然而在低并发、高容量的环境下，大key的界限可能在100kb。因此，在设计与运用Redis时，要依据业务需求与性能指标来确立合理的大key阈值。

## # 大Key问题的缺点？

- 内存占用过高。大Key占用过多的内存空间，可能导致可用内存不足，从而触发内存淘汰策略。在极端情况下，可能导致内存耗尽，Redis实例崩溃，影响系统的稳定性。
- 性能下降。大Key会占用大量内存空间，导致内存碎片增加，进而影响Redis的性能。对于大Key的操作，如读取、写入、删除等，都会消耗更多的CPU时间和内存资源，进一步降低系统性能。
- 阻塞其他操作。某些对大Key的操作可能会导致Redis实例阻塞。例如，使用DEL命令删除一个大Key时，可能会导致Redis实例在一段时间内无法响应其他客户端请求，从而影响系统的响应时间和吞吐量。
- 网络拥塞。每次获取大key产生的网络流量较大，可能造成机器或局域网的带宽被打满，同时波及其他服务。例如：一个大key占用空间是1MB，每秒访问1000次，就有1000MB的流量。
- 主从同步延迟。当Redis实例配置了主从同步时，大Key可能导致主从同步延迟。由于大Key占用较多内存，同步过程中需要传输大量数据，这会导致主从之间的网络传输延迟增加，进而影响数据一致性。
- 数据倾斜。在Redis集群模式中，某个数据分片的内存使用率远超其他数据分片，无法使数据分片的内存资源达到均衡。另外也可能造成Redis内存达到maxmemory参数定义的上限导致重要的key被逐出，甚至引发内存溢出。

## # Redis大key如何解决？

- 对大Key进行拆分。例如将含有数万成员的一个HASH Key拆分为多个HASH Key，并确保每个Key的成员数量在合理范围。在Redis集群架构中，拆分大Key能对数据分片间的内存平衡起到显著作用。
- 对大Key进行清理。将不适用Redis能力的数据存至其它存储，并在Redis中删除此类数据。注意，要使用异步删除。
- 监控Redis的内存水位。可以通过监控系统设置合理的Redis内存报警阈值进行提醒，例如Redis内存使用率超过70%、Redis的内存存在1小时内增长率超过20%等。
- 对过期数据进行定期清。堆积大量过期数据会造成大Key的产生，例如在HASH数据类型中以增量的形式不断写入大量数据而忽略了数据的时效性。可以通过定时任务的方式对失效数据进行清理。

## # 什么是热key?

通常以其接收到的Key被请求频率来判定，例如：

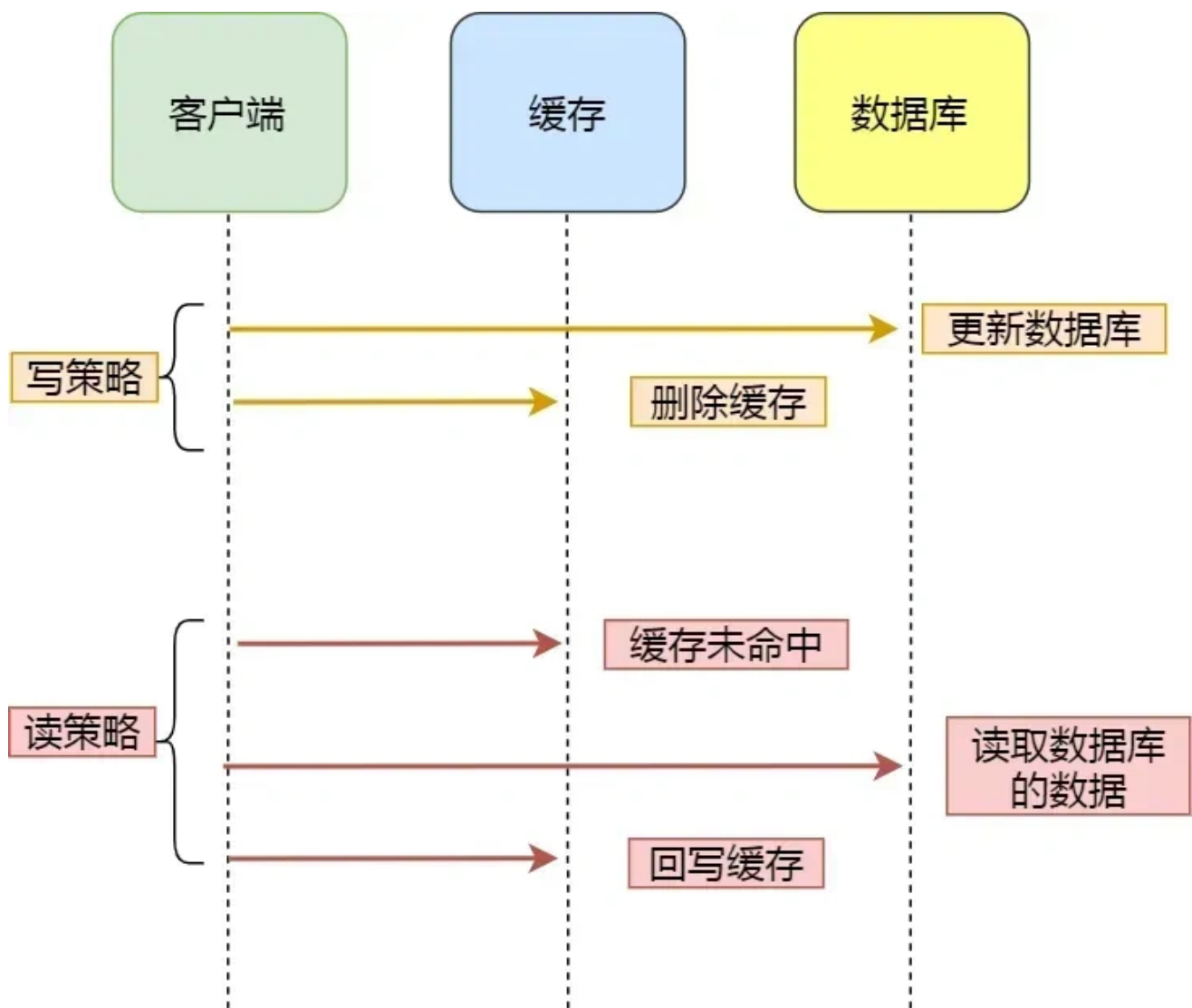
- QPS集中在特定的Key：Redis实例的总QPS（每秒查询率）为10,000，而其中一个Key的每秒访问量达到了7,000。
- 带宽使用率集中在特定的Key：对一个拥有上千个成员且总大小为1 MB的HASH Key每秒发送大量的**HGETALL**操作请求。
- CPU使用时间占比集中在特定的Key：对一个拥有数万个成员的Key（ZSET类型）每秒发送大量的**ZRANGE**操作请求。

## # 如何解决热key问题?

- 在Redis集群架构中对热Key进行复制。在Redis集群架构中，由于热Key的迁移粒度问题，无法将请求分散至其他数据分片，导致单个数据分片的压力无法下降。此时，可以将对应热Key进行复制并迁移至其他数据分片，例如将热Key foo复制出3个内容完全一样的Key并名为foo2、foo3、foo4，将这三个Key迁移到其他数据分片来解决单个数据分片的热Key压力。
- 使用读写分离架构。如果热Key的产生来自于读请求，您可以将实例改造成读写分离架构来降低每个数据分片的读请求压力，甚至可以不断地增加从节点。但是读写分离架构在增加业务代码复杂度的同时，也会增加Redis集群架构复杂度。不仅要为多个从节点提供转发层（如Proxy，LVS等）来实现负载均衡，还要考虑从节点数量显著增加后带来故障率增加的问题。Redis集群架构变更会为监控、运维、故障处理带来了更大的挑战。

## # 如何保证 redis 和 mysql 数据缓存一致性问题?

对于读数据，我会选择旁路缓存策略，如果 cache 不命中，会从 db 加载数据到 cache。对于写数据，我会选择更新 db 后，再删除缓存。



缓存是通过牺牲强一致性来提高性能的。这是由**CAP理论**决定的。缓存系统适用的场景就是非强一致性的场景，它属于CAP中的AP。所以，如果需要数据库和缓存数据保持强一致，就不适合使用缓存。

所以使用缓存提升性能，就是会有数据更新的延迟。这需要在设计时结合业务仔细思考是否适合用缓存。然后缓存一定要设置过期时间，这个时间太短、或者太长都不好：

- 太短的话请求可能会比较多的落到数据库上，这也意味着失去了缓存的优势。
- 太长的话缓存中的脏数据会使系统长时间处于一个延迟的状态，而且系统中长时间没有人访问的数据一直存在内存中不过期，浪费内存。

但是，通过一些方案优化处理，是可以**最终一致性的**。

针对删除缓存异常的情况，可以使用 2 个方案避免：

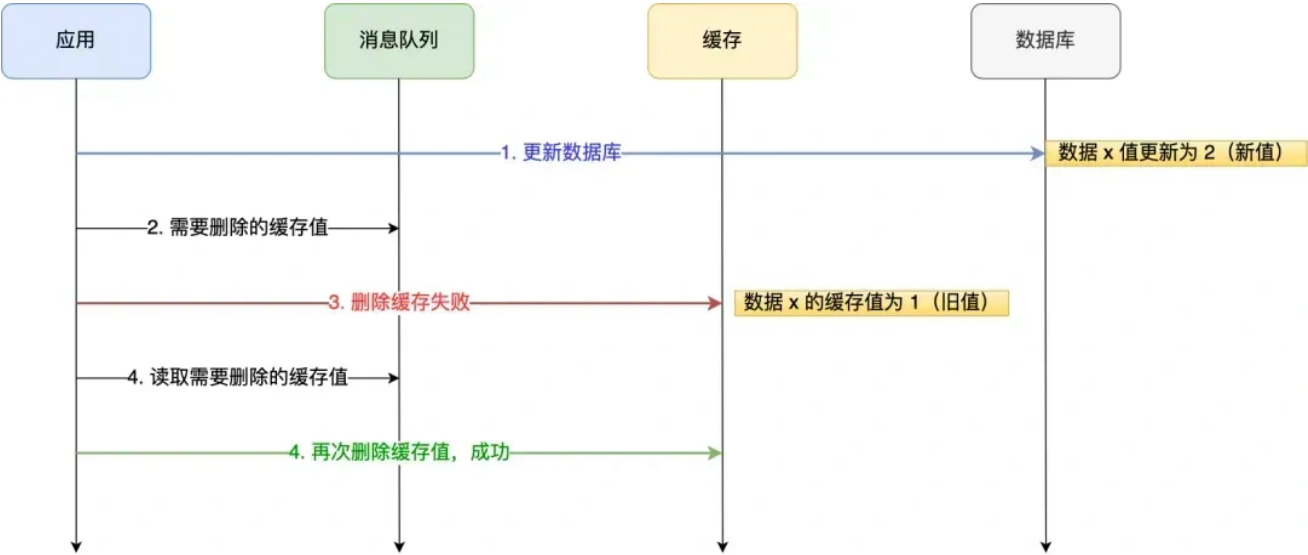
- 删除缓存重试策略（消息队列）
- 订阅 binlog，再删除缓存（Canal+消息队列）

#### 消息队列方案

我们可以引入**消息队列**，将第二个操作（删除缓存）要操作的数据加入到消息队列，由消费者来操作数据。

- 如果应用**删除缓存失败**，可以从消息队列中重新读取数据，然后再次删除缓存，这个就是**重试机制**。当然，如果重试超过的一定次数，还是没有成功，我们就需要向业务层发送报错信息了。
- 如果**删除缓存成功**，就要把数据从消息队列中移除，避免重复操作，否则就继续重试。

举个例子，来说明重试机制的过程。



重试删除缓存机制还可以，就是会造成好多业务代码入侵。

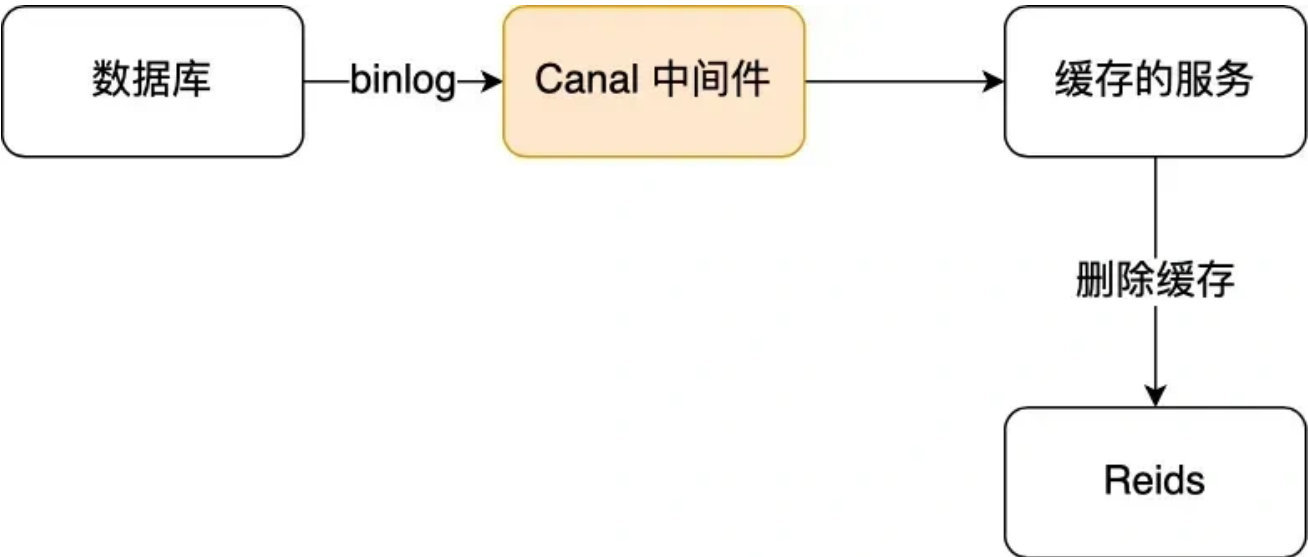
订阅 MySQL binlog，再操作缓存

「先更新数据库，再删缓存」的策略的第一步是更新数据库，那么更新数据库成功，就会产生一条变更日志，记录在 binlog 里。

于是我们就可以通过订阅 binlog 日志，拿到具体要操作的数据，然后再执行缓存删除，阿里巴巴开源的 Canal 中间件就是基于这个实现的。

Canal 模拟 MySQL 主从复制的交互协议，把自己伪装成一个 MySQL 的从节点，向 MySQL 主节点发送 dump 请求，MySQL 收到请求后，就会开始推送 Binlog 给 Canal，Canal 解析 Binlog 字节流之后，转换为便于读取的结构化数据，供下游程序订阅使用。

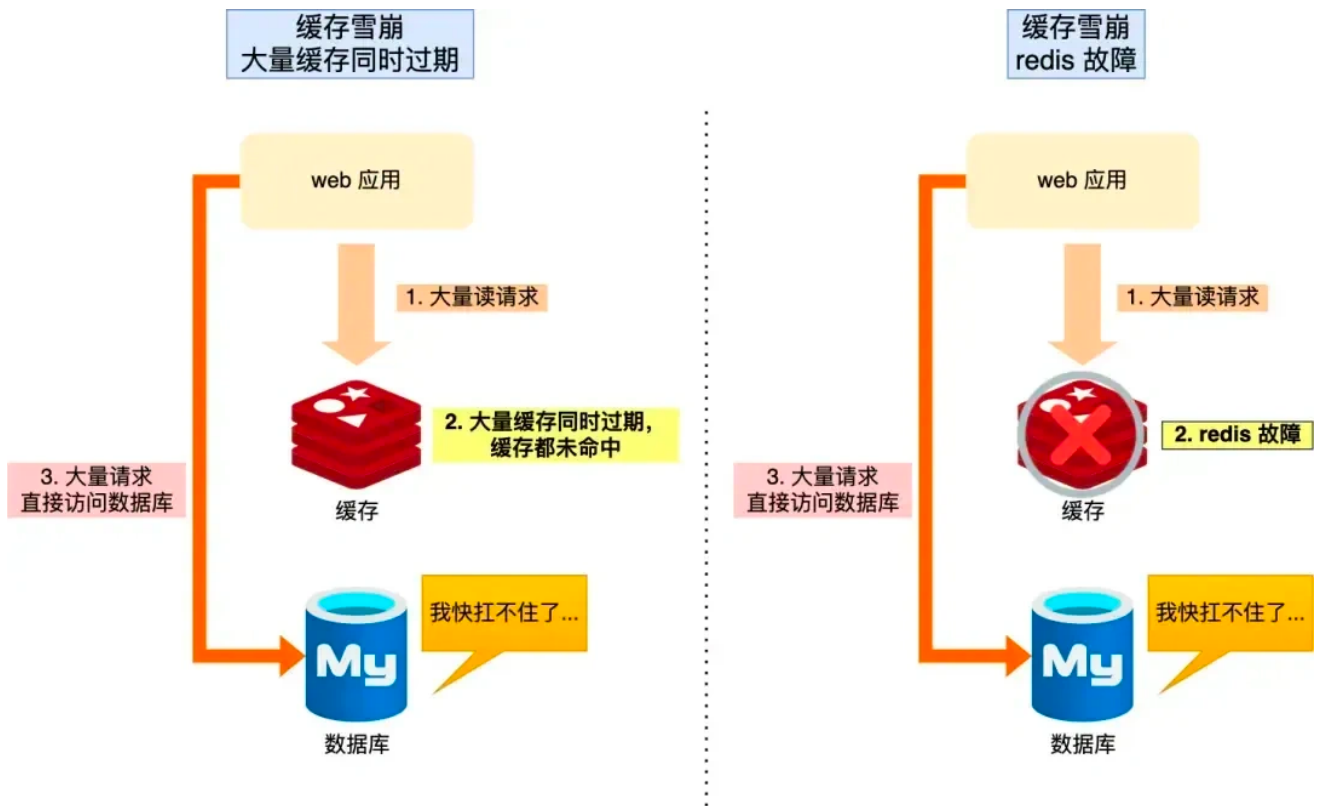
下图是 Canal 的工作原理：



将binlog日志采集发送到MQ队列里面，然后编写一个简单的缓存删除消息者订阅binlog日志，根据更新log删除缓存，并且通过ACK机制确认处理这条更新log，保证数据缓存一致性

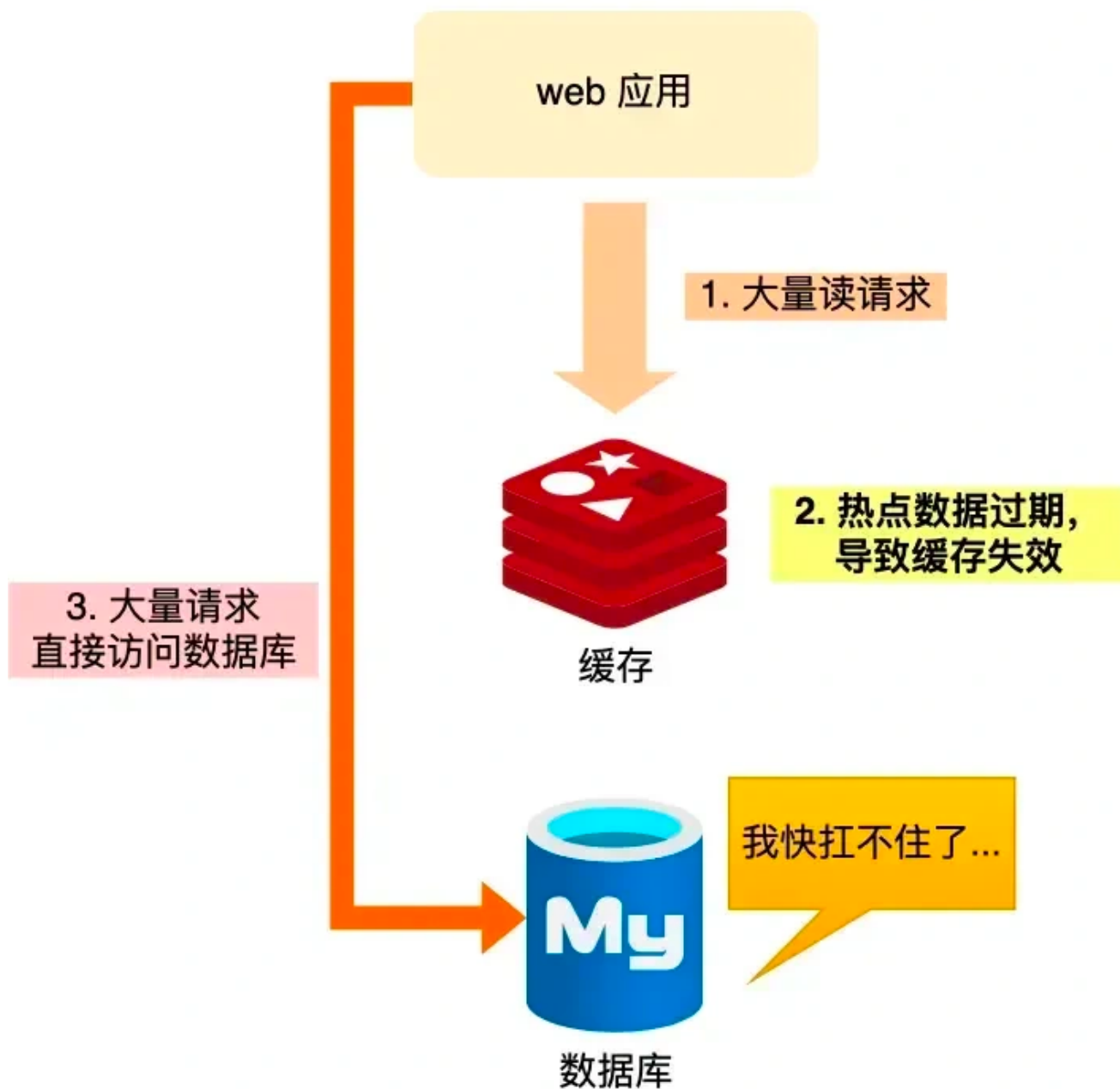
# 缓存雪崩、击穿、穿透是什么？怎么解决？

- 缓存雪崩：当**大量缓存数据在同一时间过期（失效）**或者**Redis 故障宕机**时，如果此时有大量的用户请求，都无法在 Redis 中处理，于是全部请求都直接访问数据库，从而导致数据库的压力骤增，严重的会造成数据库宕机，从而形成一系列连锁反应，造成整个系统崩溃，这就是**缓存雪崩**的问题。



- 缓存击穿：如果缓存中的**某个热点数据过期了**，此时大量的请求访问了该热点数据，就无法从缓存中读取，直接访问数据库，数据库很容易就被高并发的请求冲垮，这就是**缓存击穿**的问题。

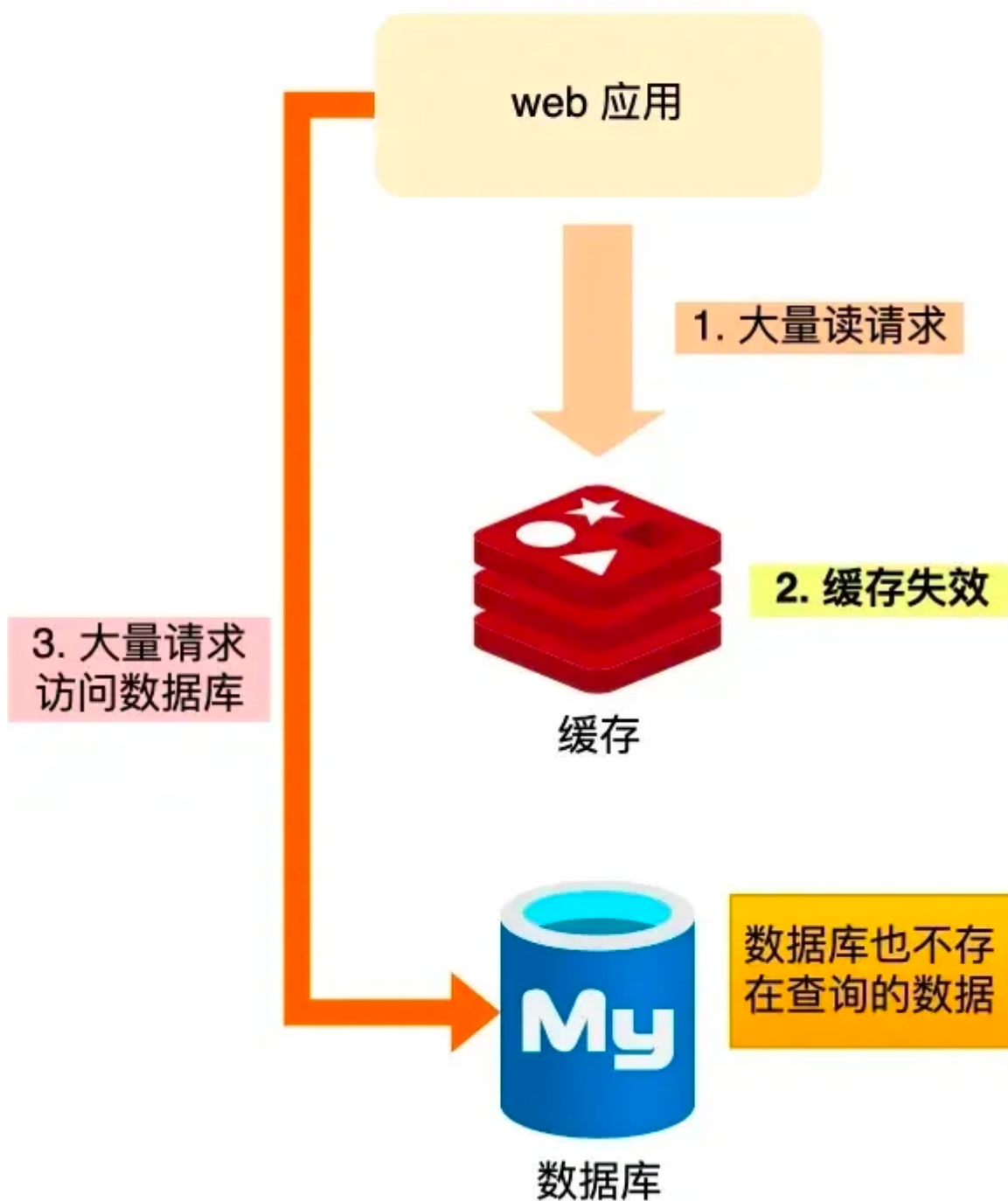
## 缓存击穿



- 缓存穿透：当用户访问的数据，**既不在缓存中，也不在数据库中**，导致请求在访问缓存时，发现缓存缺失，再去访问数据库时，发现数据库中也没有要访问的数据，没办法构建缓存数据，来服务后续的请求。那么当有大量这样的请求到来时，数据库的压力骤增，这就是**缓存穿透**的问题。



## 缓存穿透



缓存雪崩解决方案：

- 均匀设置过期时间：如果要给缓存数据设置过期时间，应该避免将大量的数据设置成同一个过期时间。我们可以在对缓存数据设置过期时间时，**给这些数据的过期时间加上一个随机数**，这样就保证数据不会在同一时间过期。

- 互斥锁：当业务线程在处理用户请求时，**如果发现访问的数据不在 Redis 里，就加个互斥锁，保证同一时间内只有一个请求来构建缓存**（从数据库读取数据，再将数据更新到 Redis 里），当缓存构建完成后，再释放锁。未能获取互斥锁的请求，要么等待锁释放后重新读取缓存，要么就返回空值或者默认值。实现互斥锁的时候，最好设置**超时时间**，不然第一个请求拿到了锁，然后这个请求发生了某种意外而一直阻塞，一直不释放锁，这时其他请求也一直拿不到锁，整个系统就会出现无响应的现象。
- 后台更新缓存：业务线程不再负责更新缓存，缓存也不设置有效期，而是**让缓存“永久有效”，并将更新缓存的工作交由后台线程定时更新**。

缓存击穿解决方案：

- 互斥锁方案，保证同一时间只有一个业务线程更新缓存，未能获取互斥锁的请求，要么等待锁释放后重新读取缓存，要么就返回空值或者默认值。
- 不给热点数据设置过期时间，由后台异步更新缓存，或者在热点数据准备要过期前，提前通知后台线程更新缓存以及重新设置过期时间；

缓存穿透解决方案：

- 非法请求的限制：当有大量恶意请求访问不存在的数据的时候，也会发生缓存穿透，因此在 API 入口处我们要判断请求参数是否合理，请求参数是否含有非法值、请求字段是否存在，如果判断出是恶意请求就直接返回错误，避免进一步访问缓存和数据库。
- 缓存空值或者默认值：当我们线上业务发现缓存穿透的现象时，可以针对查询的数据，在缓存中设置一个空值或者默认值，这样后续请求就可以从缓存中读取到空值或者默认值，返回给应用，而不会继续查询数据库。
- 布隆过滤器：我们可以在写入数据库数据时，使用布隆过滤器做个标记，然后在用户请求到来时，业务线程确认缓存失效后，可以通过查询布隆过滤器快速判断数据是否存在，如果不存在，就不用通过查询数据库来判断数据是否存在。即使发生了缓存穿透，大量请求只会查询 Redis 和布隆过滤器，而不会查询数据库，保证了数据库能正常运行，Redis 自身也是支持布隆过滤器的。

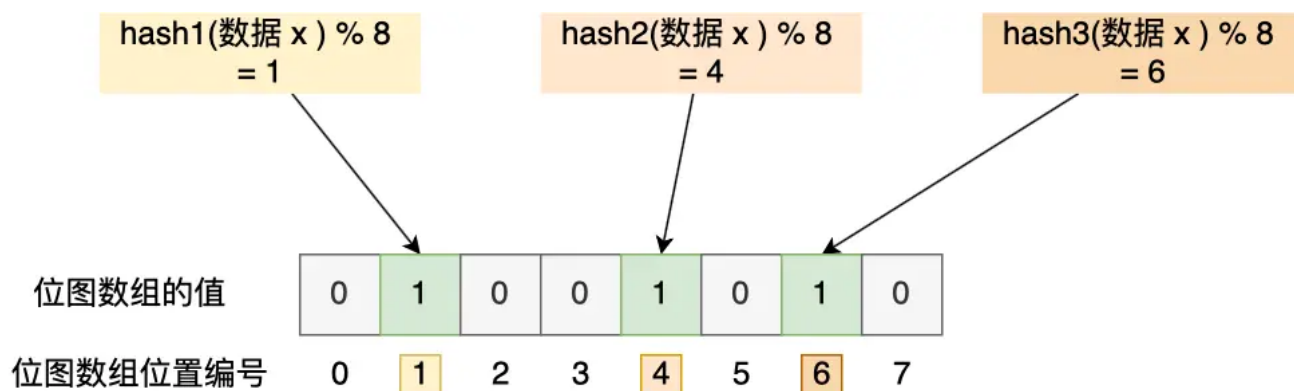
## # 布隆过滤器原理介绍一下

布隆过滤器由「初始值都为 0 的位图数组」和「N 个哈希函数」两部分组成。当我们在写入数据库数据时，在布隆过滤器里做个标记，这样下次查询数据是否在数据库时，只需要查询布隆过滤器，如果查询到数据没有被标记，说明不在数据库中。

布隆过滤器会通过 3 个操作完成标记：

- 第一步，使用 N 个哈希函数分别对数据做哈希计算，得到 N 个哈希值；
- 第二步，将第一步得到的 N 个哈希值对位图数组的长度取模，得到每个哈希值在位图数组的对应位置。
- 第三步，将每个哈希值在位图数组的对应位置的值设置为 1；

举个例子，假设有一个位图数组长度为 8，哈希函数 3 个的布隆过滤器。



在数据库写入数据 x 后，把数据 x 标记在布隆过滤器时，数据 x 会被 3 个哈希函数分别计算出 3 个哈希值，然后在这 3 个哈希值对 8 取模，假设取模的结果为 1、4、6，然后把位图数组的第 1、4、6 位置的值设置为 1。**当应用要查询数据 x 是否数据库时，通过布隆过滤器只要查到位图数组的第 1、4、6 位置的值是否全为 1，只要有一个为 0，就认为数据 x 不在数据库中。**

布隆过滤器由于是基于哈希函数实现查找的，高效查找的同时**存在哈希冲突的可能性**，比如数据 x 和数据 y 可能都落在第 1、4、6 位置，而事实上，可能数据库中并不存在数据 y，存在误判的情况。

所以，**查询布隆过滤器说数据存在，并不一定证明数据库中存在这个数据，但是查询到数据不存在，数据库中一定就不存在这个数据。**

## # 如何设计秒杀场景处理高并发以及超卖现象？

在数据库层面解决

1. 在查询商品库存时加排他锁，执行如下语句：

```
select * from goods for where goods_id=? for update
```

在事务中线程A通过select \* from goods for where goods\_id=#{id} for update语句给goods\_id为#{id}的数据行上了锁。那么其他线程此时可以使用select语句读取数据，但是如果也使用select for update语句加锁，或者使用update，delete都会阻塞，直到线程A将事务提交（或者回滚），其他线程中的某个线程排在线程A后的线程才能获取到锁。

1. 更新数据库减库存的时候，进行库存限制条件

```
update goods set stock = stock - 1 where goods_id = ? and stock >0
```

这种通过数据库加锁来解决的方案，性能不是很好，在高并发的情况下，还可能因为获取不到数据库连接或者因为超时等待而报错。

利用分布式锁

同一个锁key，同一时间只能有一个客户端拿到锁，其他客户端会陷入无限的等待来尝试获取那个锁，只有获取到锁的客户端才能执行下面的业务逻辑。

这种方案的缺点是同一个商品在多用户同时下单的情况下，会基于分布式锁串行化处理，导致没法同时处理同一个商品的大量下单的请求。

利用分布式锁+分段缓存

把数据分成很多个段，每个段是一个单独的锁，所以多个线程过来并发修改数据的时候，可以并发的修改不同段的数据

假设场景：假如你现在商品有100个库存，在redis存放5个库存key，形如：

```
key1=goods-01,value=20;
key2=goods-02,value=20;
key3=goods-03,value=20
```

用户下单时对用户id进行%5计算，看落在哪个redis的key上，就去取哪个，这样每次就能够处理5个进程请求

这种方案可以解决同一个商品在多用户同时下单的情况，但有个坑需要解决：当某段锁的库存不足，一定要实现自动释放锁然后换下一个分段库存再次尝试加锁处理，此种方案复杂比较高。

利用redis的incr、decr的原子性 + 异步队列

#### 实现思路

- 1、在系统初始化时，将商品的库存数量加载到redis缓存中
- 2、接收到秒杀请求时，在redis中进行预减库存（利用redis decr的原子性），当redis中的库存不足时，直接返回秒杀失败，否则继续进行第3步；
- 3、将请求放入异步队列中，返回正在排队中；
- 4、服务端异步队列将请求出队（哪些请求可以出队，可以根据业务来判定，比如：判断对应用户是否已经秒杀过对应商品，防止重复秒杀），出队成功的请求可以生成秒杀订单，减少数据库库存（在扣减库存的sql如下，返回秒杀订单详情）

```
update goods set stock = stock - 1 where goods_id = ? and stock > 0
```

- 5、用户在客户端申请秒杀请求后，进行轮询，查看是否秒杀成功，秒杀成功则进入秒杀订单详情，否则秒杀失败

这种方案的缺点：由于是通过异步队列写入数据库中，可能存在数据不一致，其次引用多个组件复杂度比较高

 面试鸭 mianshiya.com

# 程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

刷高频题拿高薪offer



最新的图解文章都在公众号首发，别忘记关注哦！！如果你想加入百人技术交流群，扫码下方二维码回复「加群」。



扫一扫，关注「小林coding」公众号

图解计算机基础  
认准**小林coding**

每一张图都包含小林的认真  
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」  
获取图解系列 PDF
- ② 关注公众号回复「**加群**」  
拉你进百人技术交流群